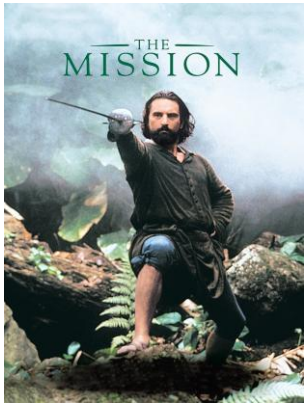
MACHINE LEARNING 2

Ciência de Dados para Negócios

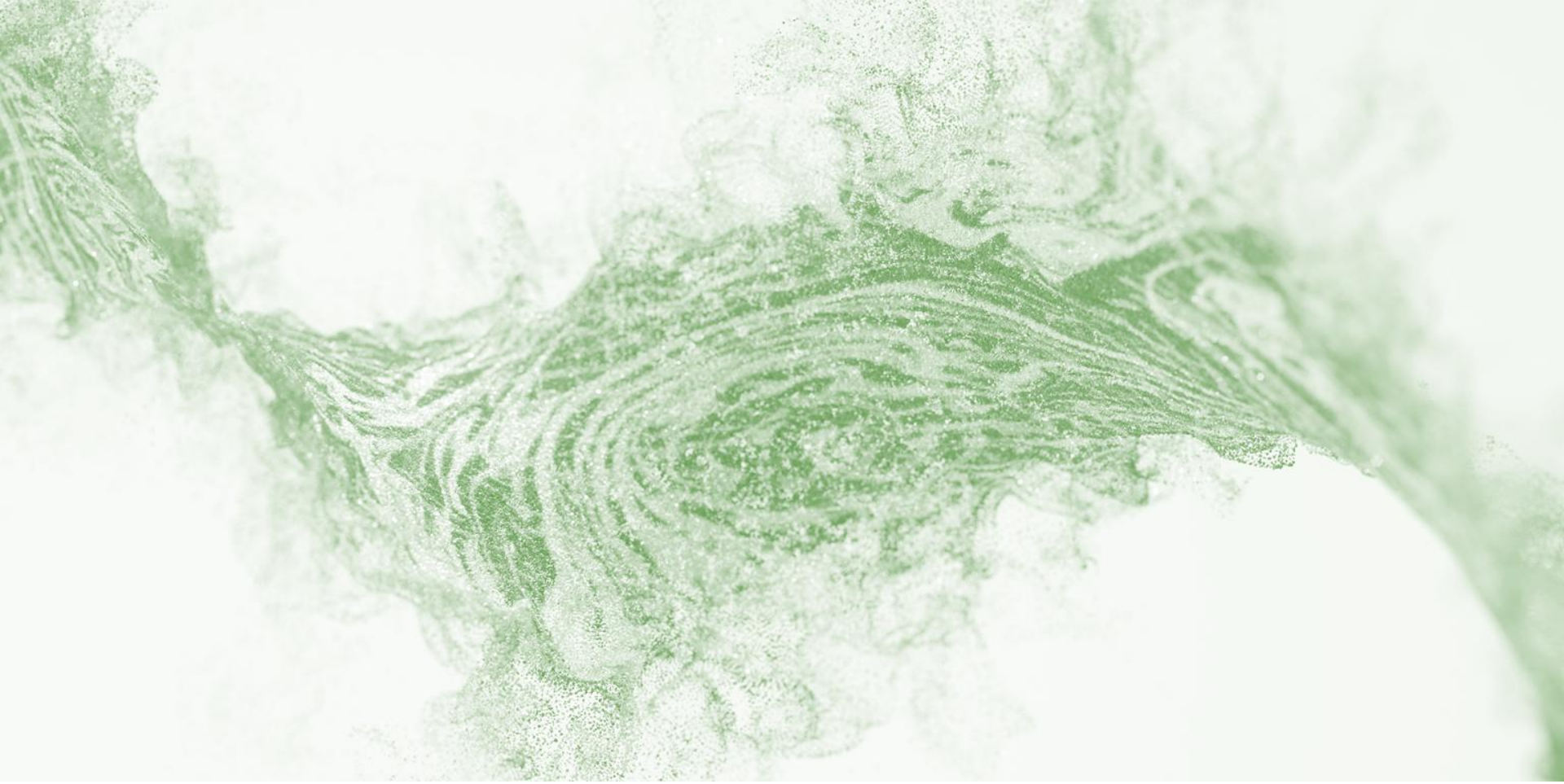
FATEC Votorantim



Prof. Luis Claudio

M. Sc. Eng. Elétrica(PUC-Rio-06)

Eng. de Computação (ITA-99)



I – INTRODUÇÃO ÀS REDES NEURAIS ARTIFICIAIS

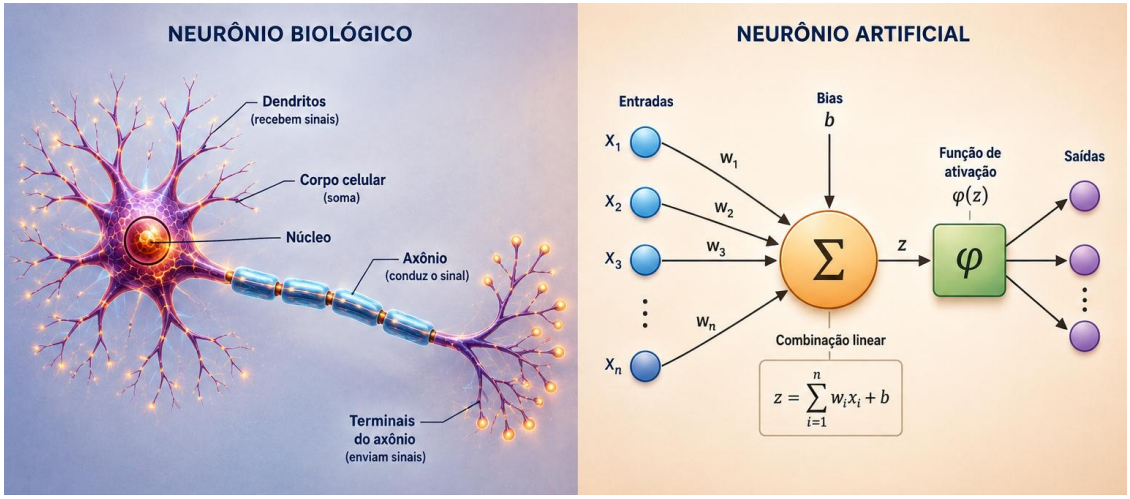
Prof. Luis Claudio
M.Sc. Eng. Elétrica(PUC-Rio-06)
Eng. de Computação (ITA-99)

O QUE É UMA REDE NEURAL

Na década de 1940, pesquisadores como **Warren McCulloch** e **Walter Pitts** tentaram responder à pergunta:

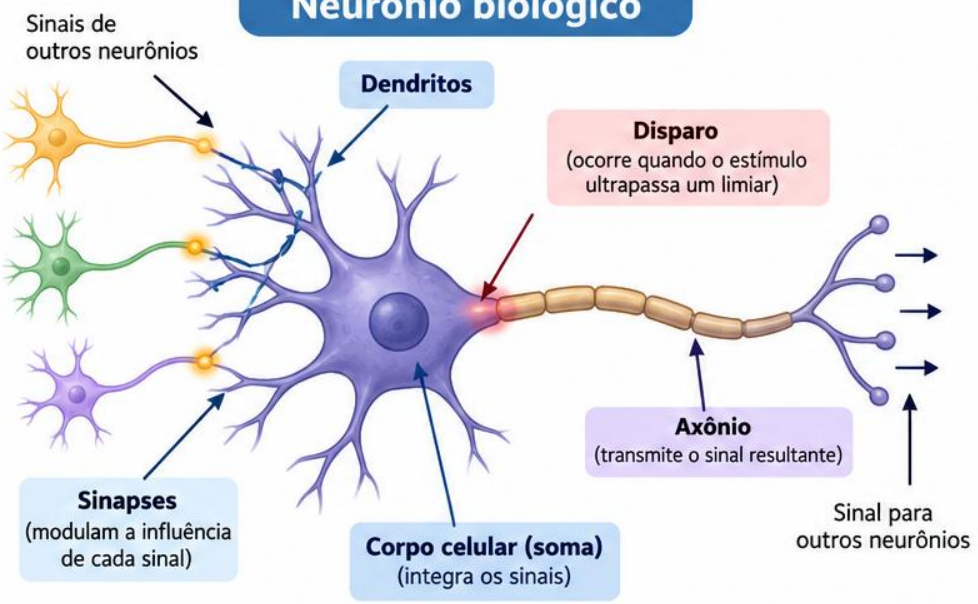
> **"Como o cérebro consegue aprender e processar informações a partir de bilhões de neurônios interconectados?"**

A ideia foi criar um modelo matemático simplificado do neurônio biológico:

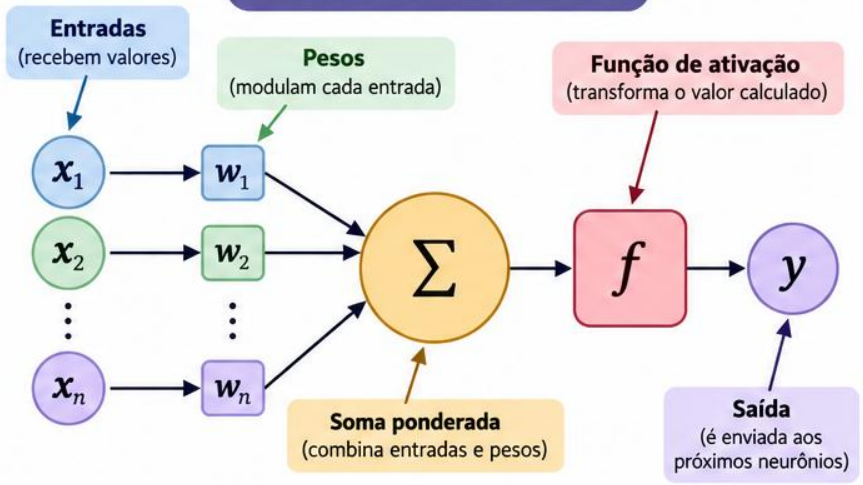


Neurônio biológico	Neurônio artificial
Dendritos recebem sinais de outros neurônios	**Entradas** $(x_1, x_2, \ldots, x_n)$ recebem valores
Sinapses modulam a influência de cada sinal	**Pesos** $(w_1, w_2, \ldots, w_n)$ modulam cada entrada
Corpo celular (soma) integra os sinais	**Soma ponderada** combina entradas e pesos
Disparo ocorre quando o estímulo ultrapassa um limiar	**Função de ativação** transforma o valor calculado
Axônio transmite o sinal resultante	**Saída** (y) é enviada aos próximos neurônios

Neurônio biológico



Neurônio artificial



$$z = \sum_{i=1}^{n} w_i x_i + b \quad \rightarrow \quad y = f(z)$$

NEURÔNIO ARTIFICIAL

O neurônio artificial é a **unidade básica de processamento** de uma rede neural. Ele recebe um conjunto de valores de entrada, realiza uma **combinação linear por meio de pesos** (*weights*) e de **um *bias***, e aplica uma **função de ativação** para produzir sua saída.

Matematicamente, o neurônio calcula a entrada líquida:

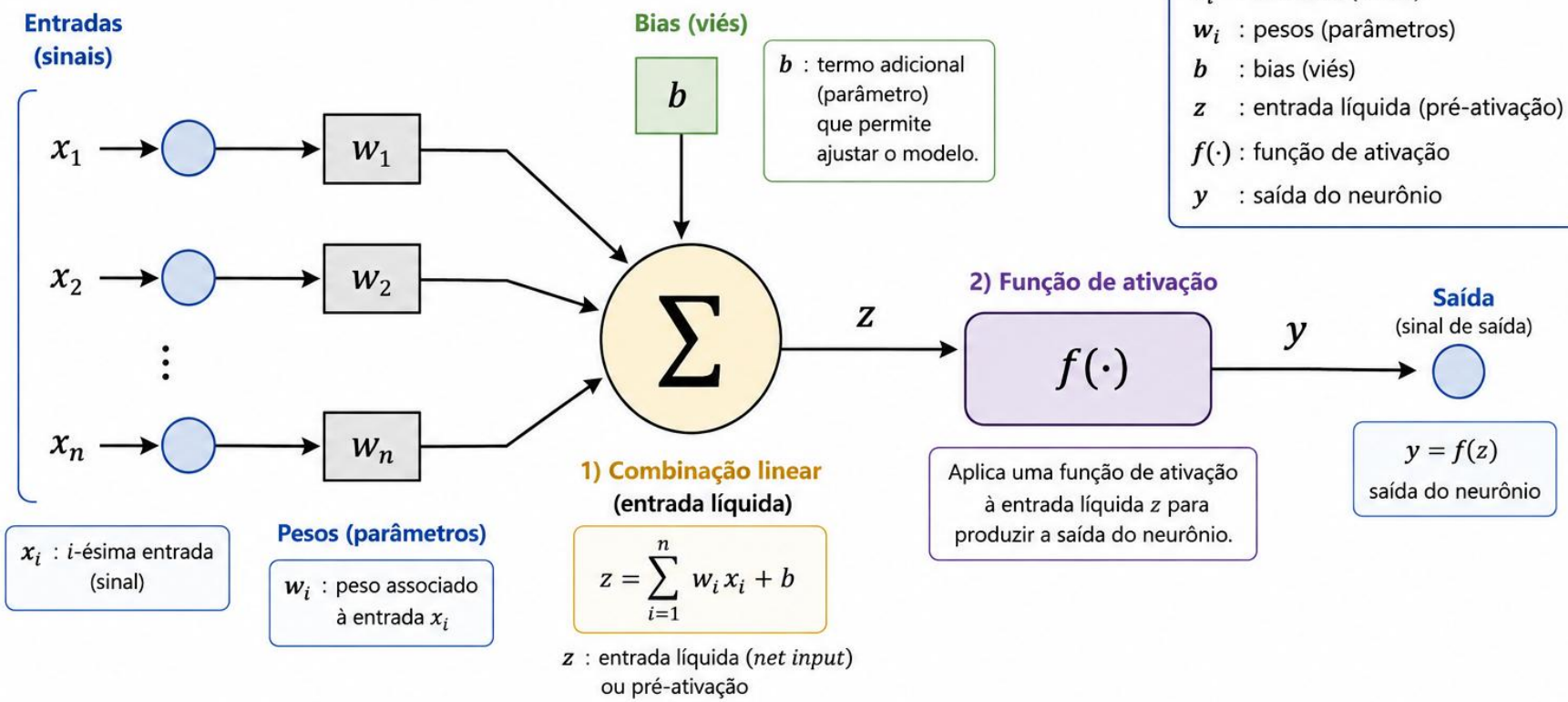
$$z = \sum_{i=1}^{n} w_i x_i + b$$

Em seguida, aplica uma função de ativação:

$$y = f(z)$$

A saída y é então encaminhada para os neurônios da camada seguinte **OU**, caso seja a última camada oculta, utilizada na camada de saída.

NEURÔNIO ARTIFICIAL



Legenda

- x_i : entradas (sinais)
- w_i : pesos (parâmetros)
- b : bias (viés)
- z : entrada líquida (pré-ativação)
- $f(\cdot)$: função de ativação
- y : saída do neurônio

b : termo adicional (parâmetro) que permite ajustar o modelo.

x_i : i-ésima entrada (sinal)

Pesos (parâmetros)

w_i : peso associado à entrada x_i

1) Combinação linear (entrada líquida)

$$z = \sum_{i=1}^{n} w_i x_i + b$$

z : entrada líquida (*net input*) ou pré-ativação

Aplica uma função de ativação à entrada líquida z para produzir a saída do neurônio.

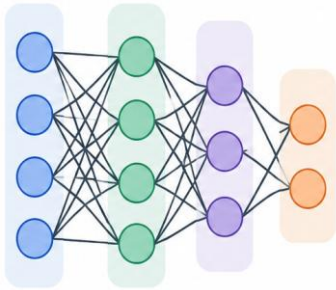
$y = f(z)$
saída do neurônio

Como funciona:

1. Cada entrada x_i é multiplicada pelo seu peso w_i.
2. Os valores ponderados são somados e o bias b é adicionado, produzindo z.
3. A função de ativação $f(\cdot)$ transforma z na saída y.
4. A saída y é enviada para os neurônios da próxima camada ou usada como saída final da rede.

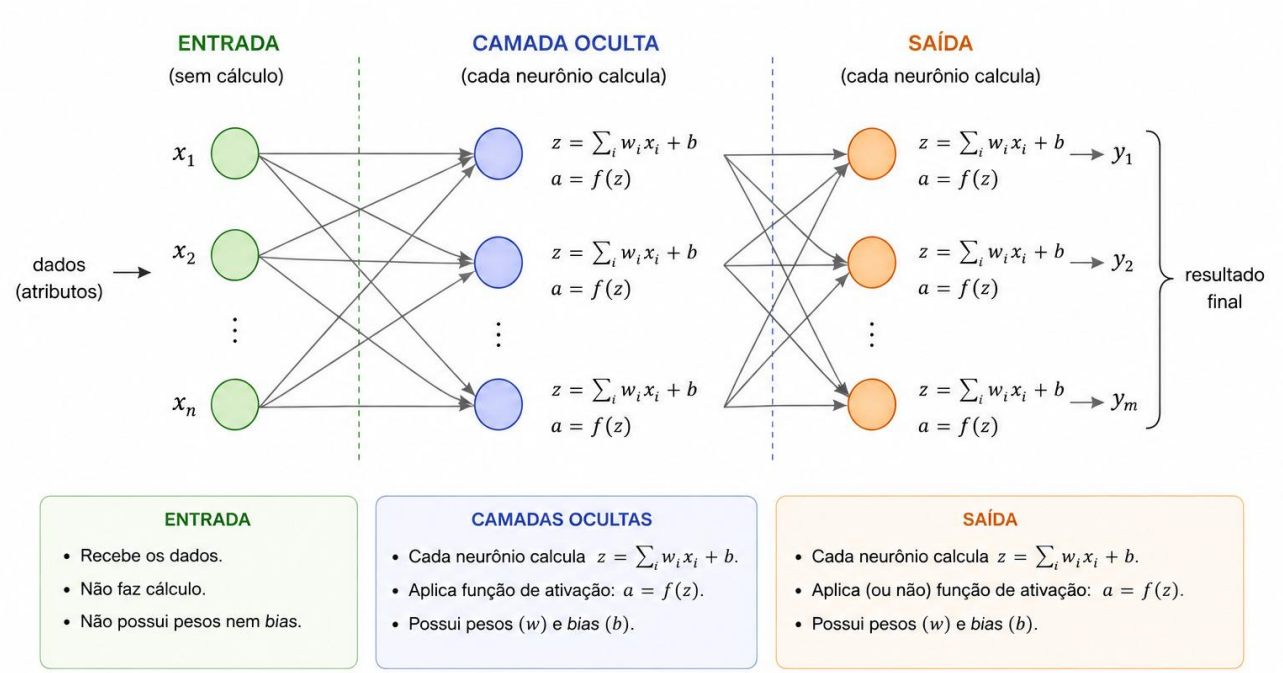
Observação histórica: Em modelos inspirados no neurônio biológico, o valor z já foi chamado de "potencial de ativação". Hoje, os termos mais comuns são entrada líquida, pré-ativação ou simplesmente z.

AS CAMADAS

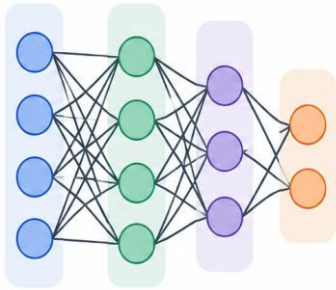


Redes neurais são arquiteturas de **aprendizagem de máquina** compostas por **neurônios artificiais organizados em camadas,** capazes de aprender representações e padrões a partir dos dados.

Em redes convencionais (como MLPs), distinguimos três tipos de camadas: **Entrada (Input),** uma ou mais **Ocultas (Hidden)** e **Saída (Output).**



A CAMADA DE ENTRADA



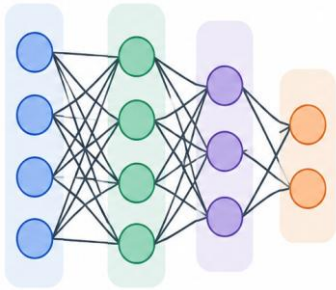
A camada de **Entrada** (*input layers*) recebe os dados de entrada (atributos), representados de **forma numérica** (dados tabulares, pixels de uma imagem etc.).

Em uma MLP tradicional (*), a camada de entrada apenas recebe os dados e os encaminha para a primeira camada oculta. Ela **não realiza processamento matemático** (**NÃO** possui pesos, bias ou função de ativação).

Por esse motivo, alguns autores **não contabilizam a camada de entrada** ao medir a **profundidade** (*depth*) da rede neural.

* Em arquiteturas mais avançadas, a camada de entrada pode ser seguida por transformações específicas, realizadas por outras camadas, como uma camada de *embedding* em problemas de PLN.

AS CAMADAS OCULTAS



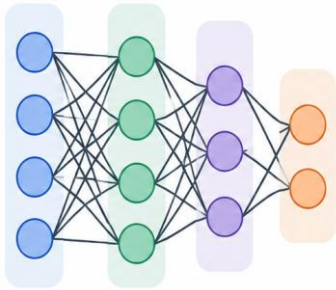
Nas camadas **Ocultas** (*hidden layers*), cada neurônio recebe as saídas da camada anterior, calcula uma combinação linear utilizando **pesos** e **bias** ($z = \sum_{i=1}^{n} w_i x_i + b$) e aplica uma **função de ativação** para produzir sua saída.

A **quantidade de camadas ocultas** e o **número de neurônios em cada camada** são **hiperparâmetros** definidos durante o projeto da rede.

As camadas ocultas quase **sempre utilizam funções de ativação**, que introduzem **não linearidade** e permitem que a rede aprenda **representações e relações complexas** a partir dos dados.

Sem funções de ativação nas camadas ocultas, sucessivas transformações lineares equivalem a uma única transformação linear, **limitando a capacidade da rede de modelar relações complexas.**

A CAMADA DE SAÍDA



Na camada de **Saída** (*output layer*) os neurônios também calculam uma combinação linear utilizando **pesos** e **bias** ($z = \sum_{i=1}^{n} w_i x_i + b$). Ou seja, ela **também tem pesos e bias aprendidos durante o treinamento**.

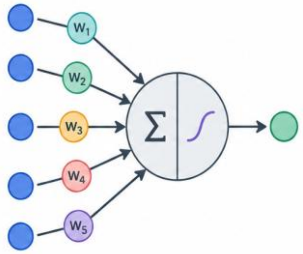
A camada de saída também usa **funções de ativação**. Mas, com o objetivo de **transformar a resposta final do modelo a uma representação mais adequada**.

A **função de ativação** depende do tipo de problema, como regressão (identidade), classificação binária (sigmoide) ou multiclasse (softmax).

Em arquiteturas tradicionais (*), a **camada de saída é única**, embora possa conter **um ou vários neurônios** também conforme a tarefa.

* Em arquiteturas mais avançadas, a rede pode possuir **múltiplas cabeças de saída (*multiple output heads*)**, geralmente compartilhando uma estrutura comum (*backbone*).

NEURÔNIO ARTIFICIAL



Entradas (x) – Conjunto de valores fornecidos ao neurônio. Na primeira camada oculta, correspondem aos atributos de entrada; nas camadas seguintes, correspondem às saídas (ativações) da camada anterior.

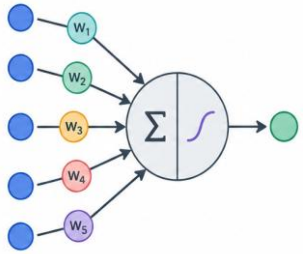
Pesos (w) – Parâmetros **aprendidos** durante o treinamento que determinam a influência de cada entrada na saída do neurônio.

Bias (b) – Termo adicional **aprendido** durante o treinamento que ajusta a combinação linear, proporcionando maior flexibilidade ao modelo.

Pré-Ativação (z) – Resultado da combinação linear entre os valores recebidos da camada anterior, seus respectivos pesos e o bias. Este valor é submetido à função de ativação.

$$z = \sum_{i=1}^{n} w_i x_i + b = w_1 x_1 + w_2 x_2 + w_3 x_3 + \cdots + w_n x_n + b$$

NEURÔNIO ARTIFICIAL



Função de Ativação (f) – Função aplicada à **pré-ativação (z)** para produzir a **ativação (a)** do neurônio. Introduz **não linearidade** no modelo, permitindo representar relações complexas entre as variáveis. O tipo de função de ativação **depende da camada e do problema que se deseja resolver**.

Ativação (a) – Resultado produzido pelo neurônio após a aplicação da função de ativação sobre **z**. Nas camadas ocultas, constitui a **saída do neurônio**, sendo enviada como entrada aos neurônios da camada seguinte.

$$\text{neurônio oculto: } z \rightarrow f(z) \rightarrow a$$

Saída do modelo ($\hat{y}$) – Resultado produzido pela **camada de saída** da rede. Representa a previsão final do modelo e sua interpretação depende do problema, podendo corresponder, por exemplo, a um valor numérico, uma probabilidade ou uma distribuição de probabilidades.

$$\text{neurônio da camada de saída: } z \rightarrow f(z) \rightarrow \hat{y}$$

Neurônio Artificial aplicado a dados tabulares

1 Dados tabulares

	idade	renda	score	...
1	25	3000	650	...
2	40	8000	700	...
3	35	5000	620	...
⋮	⋮	⋮	⋮	⋮
n	28	4500	680	...

- Cada linha = uma instância (exemplo)
- Cada coluna (de entrada) = uma característica

2 Após transformações / padronização

x_1	x_2	x_3	...	x_p
−0,45	−1,10	0,20	...	0,35
0,85	1,05	1,10	...	−0,15
0,30	−0,10	−0,30	...	0,60
⋮	⋮	⋮	⋮	⋮
−0,20	−0,60	0,40	...	0,10

Cada linha vira um vetor de **entrada**

$$x = [x_1, x_2, \ldots, x_p]$$

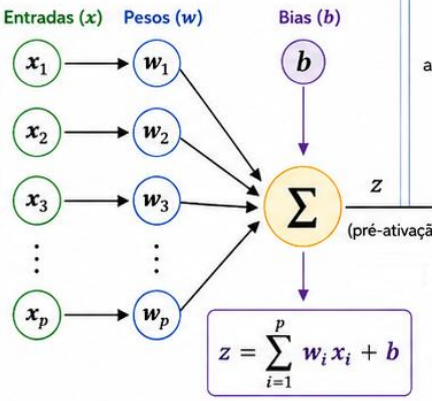
3 Entradas do neurônio (uma instância)

A linha selecionada fornece os valores numéricos que alimentam o neurônio.

$x_1 = 0{,}30$
$x_2 = -0{,}10$
$x_3 = -0{,}30$
⋮
$x_p = 0{,}60$

Entradas (x): valores numéricos da instância selecionada

4 Componentes do neurônio artificial



Pesos (w): determinam a influência de cada entrada

Bias (b): termo de ajuste adicional

Pré-ativação (z): combinação linear das entradas com pesos e *bias*

5 Função de ativação

Transforma z na ativação do neurônio.

$f(z)$

Função de ativação f: introduz não linearidade ao modelo.

6 Saída / ativação do neurônio

$a = f(z)$

a é o valor produzido pelo neurônio.

Em camadas ocultas, a é enviada à próxima camada; na camada de saída, contribui para a resposta final da rede.

Resumo do fluxo de uma instância

Linha da tabela (instância selecionada) → Uma linha da tabela de dados

Vetor de entrada x $x = [x_1, x_2, \ldots, x_p]$ → Valores padronizados da instância

Multiplicação $x_i \cdot w_i$ → Cada entrada é multiplicada pelo seu peso

Soma + bias $\sum_{i=1}^{p} w_i x_i + b$ → Soma dos produtos mais o *bias*

Pré-ativação z (entrada líquida) → Resultado da combinação linear das entradas com pesos e *bias*

Ativação $a = f(z)$ → Aplicação da função de ativação

Onde cada elemento atua:

- **Entradas (x)**: representam os valores da instância
- **Pesos (w)**: controlam a contribuição de cada entrada
- **Bias (b)**: desloca a combinação linear
- **Função de ativação f**: introduz não linearidade
- **Ativação (a)**: saída produzida pelo neurônio

BIAS

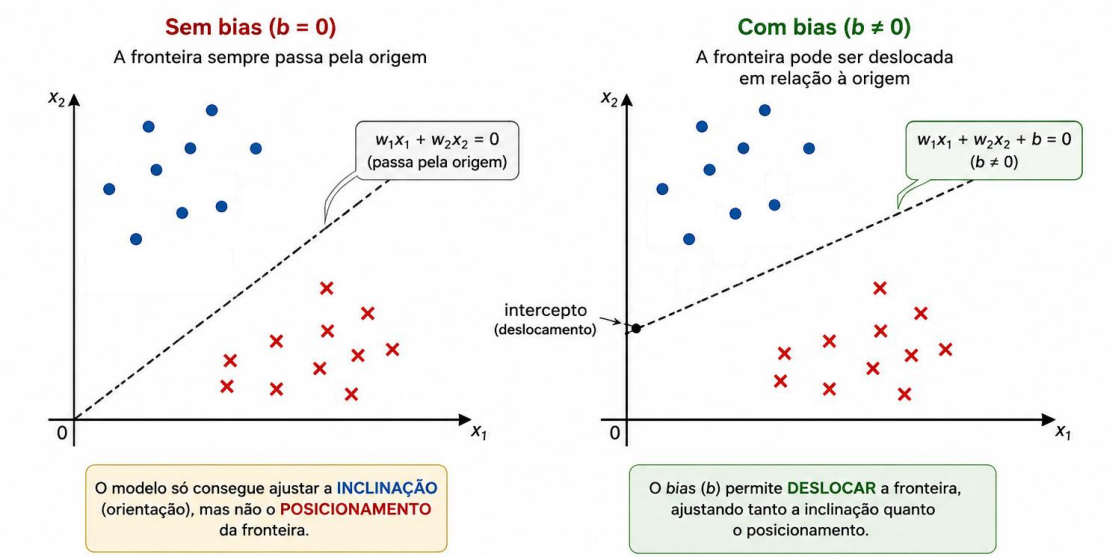
O **bias (b)** permite **deslocar a fronteira de decisão** em relação à origem.

Em duas dimensões, essa fronteira pode ser representada por uma **reta,** cuja forma é equivalente à expressão do neurônio $\mathbf{w_1x_1 + w_2x_2 + b = 0}$.
Sem o bias (b = 0), a fronteira $\mathbf{w_1x_1 + w_2x_2 = 0}$ sempre passa pela origem.
Com bias (b ≠ 0), a fronteira pode ser deslocada em relação à origem.

$$w_1x_1 + w_2x_2 + b = 0$$

$$x_2 = -\frac{w_1}{w_2}x_1 - \frac{b}{w_2}$$

$$\underbrace{x_2}_{y} = \underbrace{\left(-\frac{w_1}{w_2}\right)}_{a}\underbrace{x_1}_{x} + \underbrace{\left(-\frac{b}{w_2}\right)}_{\text{intercepto}}$$



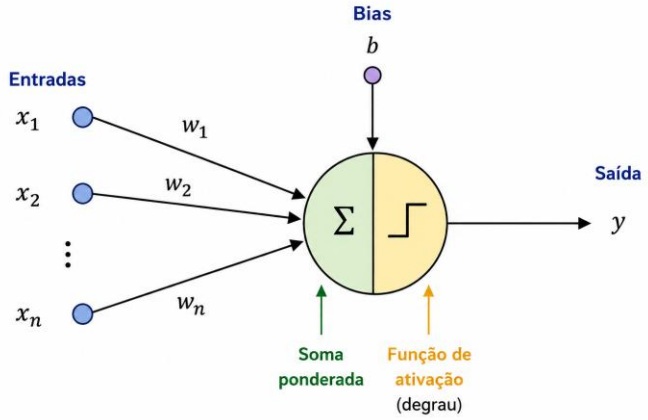


PERCEPTRON
MODELO DE ROSENBLATT

Perceptron foi proposto por **Frank Rosenblatt em 1958** e é considerado um dos primeiros modelos de neurônio artificial com **capacidade de aprendizado**. Em sua forma mais simples, pode ser representado como uma **rede neural baseada em um único neurônio**.

O Perceptron tornou-se um dos modelos mais importantes na história das Redes Neurias por introduzir o **aprendizado de máquinas.**

O **Perceptron** simples foi utilizado em problemas de **classificação binária** (**0/1, sim/não** etc.). Embora utilizasse uma **função de ativação** (*step* ou *threshold*), era capaz de resolver **problemas linearmente separáveis somente.**





PERCEPTRON
LIMITAÇÃO

O **Perceptron realiza classificação linear,** aprendendo uma **fronteira de decisão** que separa duas classes por uma **reta (2D), plano (3D) ou hiperplano (múltiplas dimensões).** Essa fronteira é definida pela **combinação linear dos atributos de entrada.**

$$z = \sum_i w_i x_i + b$$

Embora a **função degrau seja não linear,** ela apenas identifica de qual lado da fronteira está a entrada. A limitação está na capacidade de um **único neurônio,** que produz apenas uma **fronteira linear** e, portanto, resolve somente problemas **linearmente separáveis.**

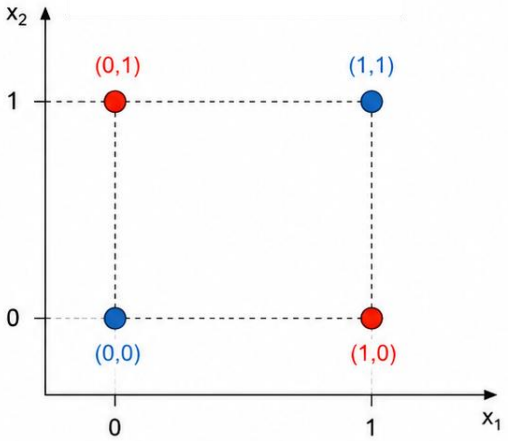
O **XOR** tornou-se o exemplo clássico dessa limitação: **não existe uma única reta capaz de separar corretamente suas duas classes.**



PERCEPTRON
LIMITAÇÃO

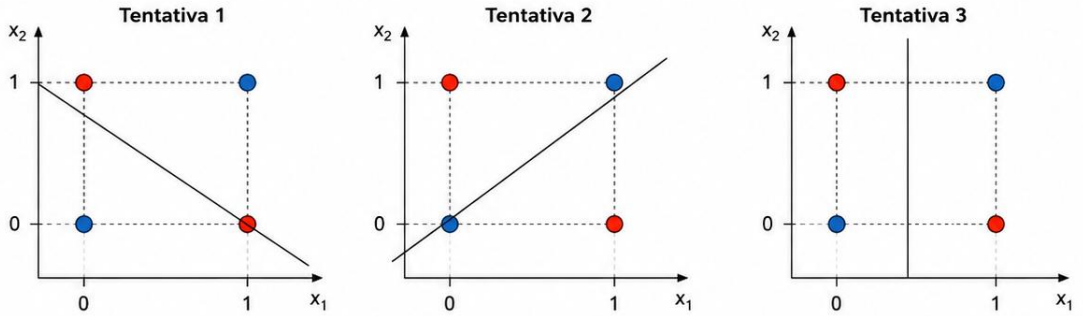
Problema XOR

x_1	x_2	x_1 XOR x_2 (saída)
0	0	0
0	1	1
1	0	1
1	1	0

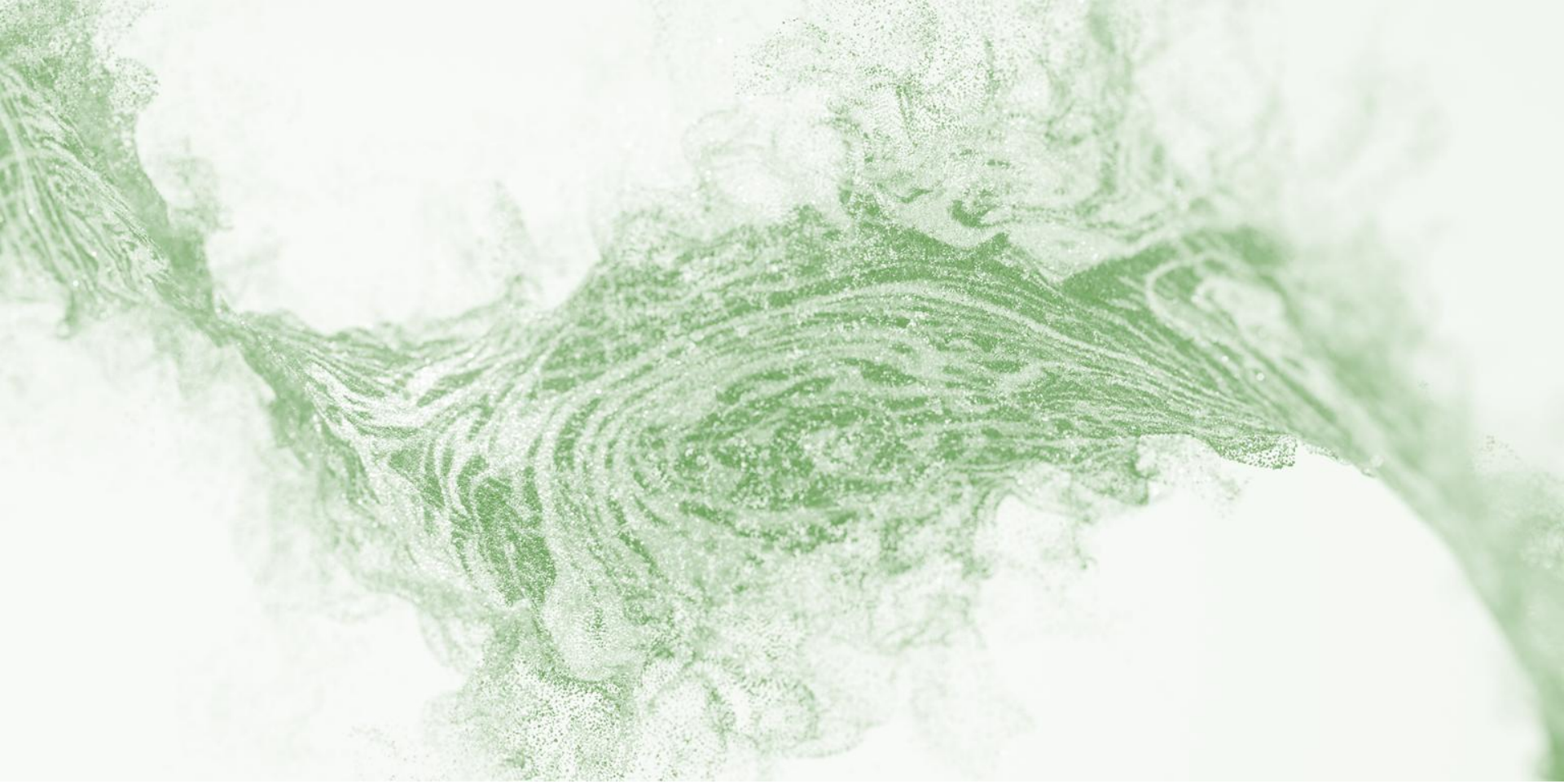


Classe 0 (saída = 0)
Classe 1 (saída = 1)

Por que o perceptron (classificação linear) não resolve?



✗ Nenhuma reta (ou hiperplano) consegue separar as classes corretamente.



II – TREINAMENTO DE REDES NEURAIS

Prof. Luis Claudio
M.Sc. Eng. Elétrica(PUC-Rio-06)
Eng. de Computação (ITA-99)

HIPERPARÂMETROS

Parâmetros

São **aprendidos** pela rede durante o treinamento. Representam **o que** o modelo aprende a partir dos dados.

- Exemplos: **pesos (weights)** e **vieses (biases)**.

Hiperparâmetros

São **definidos pelo cientista de dados**.

Determinam **como** e em **quais condições** o aprendizado ocorrerá.

- Exemplos: número de camadas (“profundidade”), número de neurônios por camada (“largura”), função de ativação, tamanho de lote (*batch size*), número de épocas, função de perda (*loss function*), otimizador e taxa de aprendizado (*learning rate*) entre outras.

HIPERPARÂMETROS
ARQUITETURA VERSUS TREINAMENTO

Embora não seja uma divisão canônica, podemos dividir em:

Hiperparâmetros de Arquitetura

São paramentos que definem **como a rede neural é estruturada**:

- Número de camadas ocultas
- Número de neurônios por camada oculta
- Função de ativação

Hiperparâmetros de Treinamento

São parâmetros que definem **como o aprendizado acontecerá**:

- Tamanho do lote (*batch size*)
- Número de épocas (*epochs*)
- Função de perda (*loss function*)
- Otimizador
- Taxa de aprendizado (*learning rate*)


ReLU / tanh /
sigmoid
HIPERPARÂMETROS
DE ARQUITETURA
(estrutura da rede)



HIPERPARÂMETROS
QTD. DE CAMADAS OCULTAS

Não existe uma regra que determine o número ideal de camadas ocultas.

A escolha inicial depende principalmente da **complexidade do problema** e deve ser **avaliada experimentalmente**.

Em geral, quanto **mais camadas, maior a capacidade** da rede de construir representações mais complexas, mas também **maior o custo computacional**.

Na pratica...

- Para problemas tabulares tradicionais, **1 a 3 camadas ocultas** costumam ser um ponto de partida razoável.
- Redes com dezenas ou centenas de camadas estão normalmente associadas a arquiteturas e problemas muito mais complexos (LLMs, por exemplo).



HIPERPARÂMETROS
QTE. DE NEURÔNIOS POR CAMADA

Diferentemente das camadas de entrada e saída, cuja quantidade de neurônios está diretamente relacionada, respectivamente, à dimensionalidade dos dados e ao tipo de problema, também **não existe uma regra que determine a quantidade ideal de neurônios nas camadas ocultas.**

Aqui também, a escolha inicial é uma **hipótese de arquitetura**, posteriormente validada experimentalmente.

Na prática...

- É comum utilizar valores como **32, 64, 128, 256**, por convenção e também por eficiência computacional (legado).
- A arquitetura pode manter, aumentar ou reduzir a quantidade de neurônios entre as camadas. Um **afunilamento progressivo** é uma estratégia bastante utilizada (mas existem outras, como “V”, “W” etc.).
- Mais neurônios aumentam a **capacidade de representação** da rede, mas também aumentam o **número de parâmetros e o custo computacional**.



HIPERPARÂMETROS
FUNÇÕES DE ATIVAÇÃO

As funções de ativação (**f**) são aplicadas ao valor de pré-ativação (**z**) para gerar o valor de ativação (**a**).

$$f(z) \rightarrow a$$

Nas **camadas ocultas**, sua principal função é **introduzir não linearidade**, permitindo que a rede aprenda relações e fronteiras de decisão complexas, que não poderiam ser representadas por sucessivas transformações lineares.

Entre as principais funções de ativação estão:

- Sigmoid
- Tanh (Tangente Hiperbólica)
- ReLU

Há diversas outras, como **Leaky ReLU, ELU, GELU, SiLU/Swish e Softplus.**



HIPERPARÂMETROS
FUNÇÕES LINEARES

By the way, uma **função linear** é uma combinação do tipo:

$$f(x_1, \ldots, x_n) = a_1 x_1 + \cdots + a_n x_n$$

Logo:

- cada variável aparece em **expoente 1**;
- não há produto entre variáveis, como $x_1 x_2$;
- não há potências como x^2, x^3;
- não há $\sqrt{x}$, $1/x$, e^x, $\log(x)$, $\sin(x)$, etc.;
- preserva a linearidade:

$$f(x+y) = f(x) + f(y), \qquad f(cx) = cf(x)$$

Exemplos:

$2x_1 - 3x_2$	**é linear**
$2x_1 x_2$; $x_1^2 + x_2$; $\sin(x_1) + x_2$	**não são lineares**



FUNÇÃO DE ATIVAÇÃO
SIGMOIDE

A função sigmoide recebe qualquer valor de **z entre -∞ e +∞** e produz uma saída entre **0 e 1**. É uma função **não linear,** contínua e com formato de "S" que produz uma **limitação/compressão** da faixa das ativações.

É **centrada em 0,5** (para **z = 0,** a saída é **0,5**); para **z ≈ 2,2,** a saída é aproximadamente **0,9**; para **z ≈ -2,2,** aproximadamente **0,1.**

$$f(z) = \frac{1}{1 + e^{-z}}$$

Para valores maiores de |z|, a função entra em uma região de **saturação,** na qual grandes alterações em z produzem pequenas alterações na saída. Essa saturação pode provocar o problema de ***vanishing gradiente*** (gradiente que desaparece), dificultando o treinamento de redes mais profundas*.

* Na retropropagação, para calcular o **gradiente da função de perda em relação aos pesos,** usamos a **regra da cadeia,** que envolve as **derivadas das funções de ativação**. Por isso a saturação é problemática: uma curva quase horizontal gera derivada ≈ 0 e o gradiente tende a zero, prejudicando a atualização dos pesos (em outras palavras "a rede para de aprender").

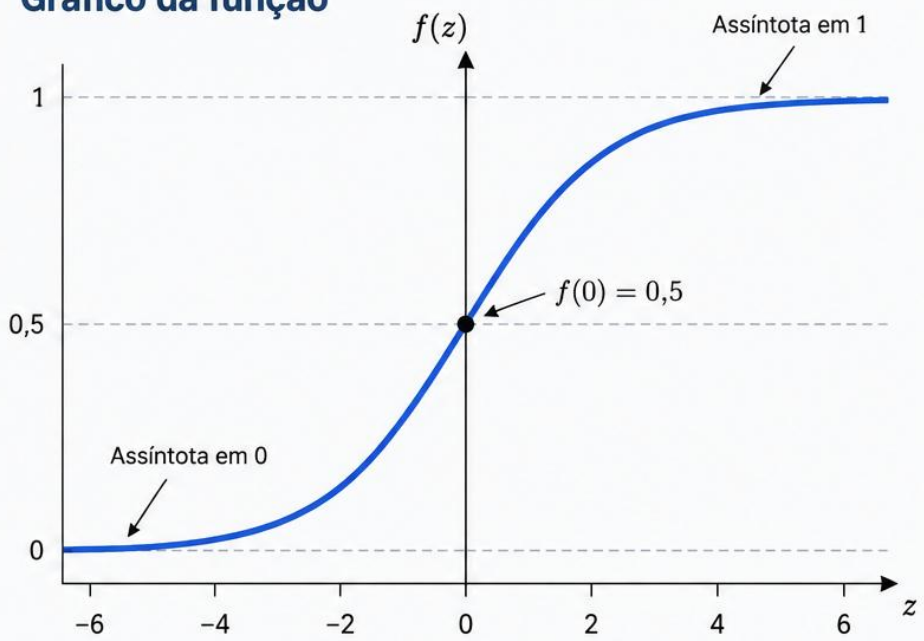
Função Sigmoide

$$f(z) = \frac{1}{1 + e^{-z}}$$

Principais características

- Recebe qualquer valor de z (de $-\infty$ a $+\infty$) e produz uma saída entre 0 e 1.
- É uma função não linear, contínua e com formato de "S".
- Para $z = 0$, a saída é 0,5.
- Para valores elevados de $|z|$, a função entra em uma região de saturação.
- A saturação pode provocar o problema de vanishing gradient, dificultando o treinamento de redes mais profundas.
- Atualmente, é pouco utilizada em camadas ocultas, sendo amplamente substituída pela ReLU e suas variantes.

Gráfico da função



Exemplos de valores

z	−4	−2	0	2	4
$f(z)$	0,018	0,119	0,500	0,881	0,982



FUNÇÃO DE ATIVAÇÃO
TANGENTE HIPERBÓLICA

A função tanh recebe qualquer valor de **z entre -∞ e +∞** e produz uma saída entre **-1 e 1**. Também é **não linear**, contínua e com formato de "S" e que também produz uma **limitação/compressão** da faixa das ativações.

É **centrada em 0** (para **z = 0**, a saída é **0**); para **z ≈ 1,47**, a saída é aproximadamente **0,9**; para **z ≈ -1,47**, aproximadamente **-0,9**.

$$f(z) = \tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

Para valores maiores de |z|, também apresenta **saturação**, aproximando-se de -1 ou 1. Ou seja, também está sujeita ao ***vanishing gradient***.

Uma vantagem da tanh em relação à sigmoid é produzir saídas centradas em zero, **preservando valores negativos e positivos**.

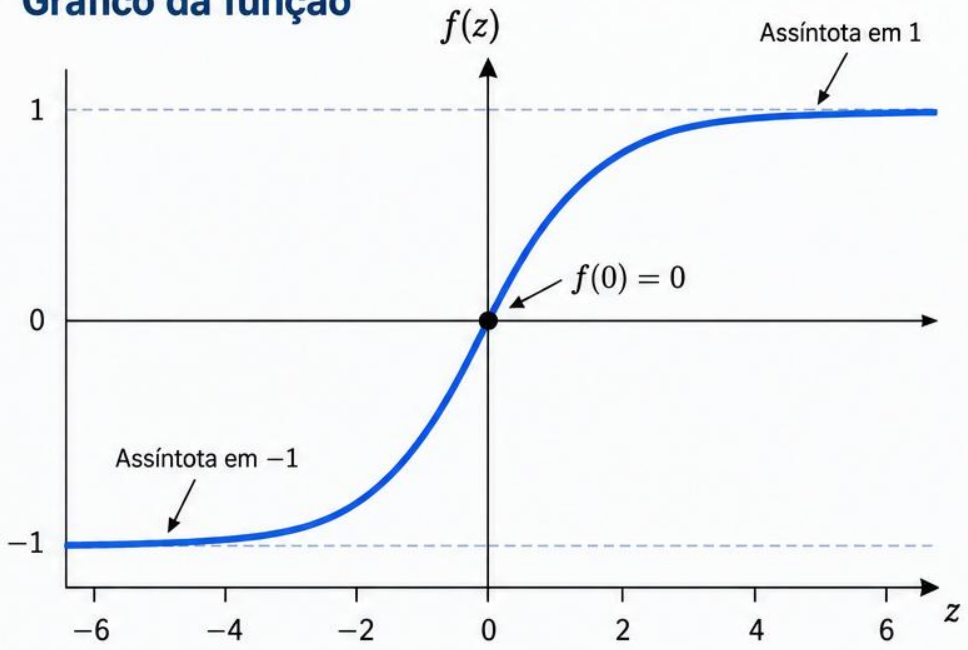
Função Tangente Hiperbólica (*Tanh*)

$$f(z) = \tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

Principais características

- Recebe qualquer valor de z (de $-\infty$ a $+\infty$) e produz uma saída entre -1 e 1.
- É uma função não linear, contínua e com formato de "S".
- É simétrica em relação à origem: $tanh(-z) = -tanh(z)$.
- Para $z = 0$, a saída é 0.
- Para valores elevados de $|z|$, a função entra em uma região de saturação (próxima de -1 ou 1).
- Também pode sofrer com o problema de vanishing gradient, embora em menor grau que a função sigmoide.
- Foi amplamente utilizada em redes neurais, sendo hoje menos comum devido ao melhor desempenho da ReLU e suas variantes.

Gráfico da função



Exemplos de valores

z	−4	−2	0	2	4
$tanh(z)$	−0,999	−0,964	0,000	0,964	0,999



HIPERPARÂMETROS
FUNÇÃO RELU

ReLU (*Rectified Linear Unit*) é atualmente uma das funções de ativação mais utilizadas em **camadas ocultas**.

Seu comportamento é **extremamente simples**:

- Para **z ≤ 0**, a saída é **0**;
- Para **z > 0**, a saída é o **próprio z**.

$$f(z) = \max(0, z)$$

Apesar de sua simplicidade, é **não linear**, pois apresenta uma mudança de comportamento em **z = 0**.

- Em **z=0**, a derivada **não é matematicamente definida**; nas implementações computacionais, adota-se convencionalmente um valor para permitir o treinamento.
- Para **z > 0**, sua derivada é **1**, evitando a saturação observada na sigmoide. Isso contribui para reduzir o problema de ***vanishing gradient***.

Como limitações, a ReLU **não limita superiormente as ativações**, que podem assumir valores muito elevados e contribuir para instabilidades no treinamento. Além disso, para z<0, sua derivada é 0, podendo ocorrer o problema conhecido como **dying ReLU**. Variantes como **Leaky ReLU** introduzem uma pequena inclinação para valores negativos, buscando reduzir este último problema.

ReLU — Rectified Linear Unit

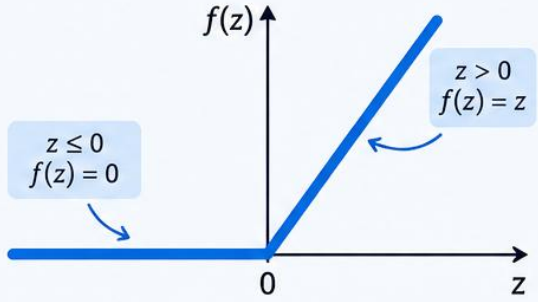
(Unidade Linear Retificada)

- Uma das funções de ativação mais utilizadas em **camadas ocultas.**
- Simples e eficiente.
- É **não linear**, pois muda o comportamento em z = 0.
- Para z > 0, sua derivada é 1, evitando a saturação e ajudando a reduzir o problema de **vanishing gradient.**

Definição

$$f(z) = \max(0, z)$$

Gráfico



Exemplos de valores

z	−4	−2	0	2	4
$f(z)$	0	0	0	2	4
$f'(z)$	0	0	(indef.)	1	1

Obs.: em z = 0, a derivada não é definida. Em implementações práticas, costuma-se usar 0 ou 1.

Vantagens

- Não satura para z > 0 (derivada = 1)
- Ajuda a reduzir o vanishing gradient
- Computacionalmente simples e eficiente
- Funciona bem em redes profundas

Limitações

- Derivada = 0 para z < 0
- Pode causar o problema de "dying ReLU" (neurônios que param de ativar)

Em resumo: a ReLU é simples, eficiente e evita a saturação na região positiva, o que facilita o treinamento de redes profundas. Por isso, é hoje a função de ativação mais usada em camadas ocultas.



HIPERPARÂMETROS
FUNÇÃO RELU

A **ReLU tornou-se uma função padrão para camadas ocultas,** especialmente em redes profundas, porque é **simples, computacionalmente barata e reduz o problema** de *vanishing gradient* na região positiva.

$$f(z) = \max(0, z)$$

Mas hoje o cenário é mais diversificado:

- **ReLU e variantes** → muito comuns em CNNs e diversas redes *feedforward*.
- **GELU** → tornou-se muito comum em **Transformers.**
- **SiLU/Swish** → utilizada em arquiteturas modernas, inclusive redes de visão.



HIPERPARÂMETROS
FUNÇÕES DE ATIVAÇÃO NA SAÍDA

Na **camada de saída**, sua principal função é **adequar a saída da rede ao tipo de problema que está sendo modelado e ao formato esperado da resposta**.

Dessa forma, a função de ativação pode determinar características da saída, como seu **intervalo de valores e sua interpretação**.

A escolha da função de ativação na camada de saída está, portanto, diretamente relacionada ao **tipo de problema**, devendo também ser **compatível com a função de perda** utilizada durante o treinamento.



HIPERPARÂMETROS
FUNÇÕES DE ATIVAÇÃO NA SAÍDA

A escolha da função de ativação e da quantidade de neurônios da **camada de saída** depende do tipo de problema:

- **Regressão:** geralmente **1 neurônio** com função **Identidade (Linear)**. A saída pode assumir valores contínuos sem limitação de intervalo.
- **Classificação Binária:** geralmente **1 neurônio** com **Sigmoid**. A saída varia entre **0 e 1** e pode ser interpretada como a **probabilidade da classe positiva**.
- **Classificação Multiclasse:** geralmente **1 neurônio por classe** com **Softmax**. As saídas representam probabilidades associadas às classes e **somam 1**.
- **Classificação Multilabel:** geralmente **1 neurônio por classe**, com **Sigmoid aplicada independentemente a cada neurônio**. Cada classe é tratada como um problema de **"Sim ou Não"**, permitindo **múltiplas classes simultaneamente**. **Exemplo:** uma imagem pode conter **pessoa, cachorro e carro** ao mesmo tempo.

MULTICLASSE (Softmax)

Apenas uma classe é escolhida.
As probabilidades somam 1.

Exemplo:
Qual **animal** aparece na imagem?



Classe	Softmax (probabilidade)
Cachorro	**0,91**
Gato	0,06
Cavalo	0,03

Soma = 1,00

➜ **Resultado: Cachorro**

Características:
Softmax impõe competição entre as classes: apenas **uma** será a escollhida.

MULTILABEL (Sigmoid)

Várias classes podem ser atribuídas simultaneamente.
Cada saída é independente (não somam 1).

Exemplo:
Quais **objetos** estão presentes na imagem?



Classe	Sigmoid (probabilidade)
Pessoa	**0,96**
Cachorro	**0,91**
Carro	**0,83**
Árvore	**0,12**

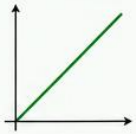
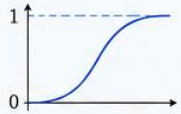
Não somam 1
(saídas independentes)

➜ **Resultado: Pessoa + Cachorro + Carro**

Características:
Cada classe é tratada como um problema de **"Sim ou Não"**.
Várias classes podem ter probabilidade alta ao mesmo tempo.

Resumo: **Multiclasse** = escolhas mutuamente exclusivas | **Multilabel** = rótulos independentes que podem coexistir.

Configuração da Camada de Saída em Redes Neurais

Tipo de Problema	Neurônios na Saída	Função de Ativação	Saída Produzida	Interpretação	Exemplo
Regressão Prever um ou mais valores contínuos (reais).	**1 (geralmente) ou mais** $\bigcirc \longrightarrow y_1$ $\bigcirc \longrightarrow y_2$ $\vdots \quad \vdots$ $\bigcirc \longrightarrow y_k$ (1 neurônio para cada variável-alvo)	**Linear (identidade)**  $f(z) = z$	**Valores reais (sem restrição)** Ex.: $y \in (-\infty, +\infty)$	Cada neurônio produz um valor real que pode assumir qualquer valor (no intervalo real).	**Exemplos:** • Preço de um imóvel • Temperatura • LTV • Prever temperatura e umidade (2 saídas) 
Classificação Binária Prever se pertence à classe positiva (1) ou negativa (0).	**1** $\bigcirc \longrightarrow y$	**Sigmoid**  $f(z) = \frac{1}{1+e^{-z}}$	**Uma probabilidade entre 0 e 1** Ex.: $y \in [0, 1]$	Representa a probabilidade da classe positiva (1). Usa-se um limiar (ex.: 0,5) para decidir a classe.	**Exemplos:** • Spam / Não spam • Fraude / Não fraude • Cliente vai comprar / Não vai comprar 
Classificação Multiclasse Existem várias classes possíveis, mas cada exemplo pertence apenas a uma delas.	**1 por classe (k classes)** $\bigcirc \longrightarrow z_1$ $\bigcirc \longrightarrow z_2$ $\vdots \quad .$ $\bigcirc \longrightarrow z_k$	**Softmax** $f(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}$ (aplicada ao vetor z)	**Distribuição de probabilidades (soma = 1)** Ex.: $[0, 0.15, 0.70, 0.15]$ $(\sum_i p_i = 1)$	A classe prevista é aquela com a maior probabilidade. As probabilidades de todas as classes somam 1.	**Exemplos:** • Tipo de animal: gato, cachorro ou cavalo • Meio de transporte: carro, ônibus ou avião 
Classificação Multirrótulo (Multi-label) Um exemplo pode pertencer a várias classes ao mesmo tempo.	**1 por classe (k classes)** $\bigcirc \longrightarrow y_1$ $\bigcirc \longrightarrow y_2$ $\vdots \quad .$ $\bigcirc \longrightarrow y_k$	**Sigmoid** (em cada neurônio)  $f(z) = \frac{1}{1+e^{-z}}$	**Probabilidade independente para cada classe** Ex.: $[0.95, 0.82, 0.10, 0.67]$ (não somam 1)	Cada neurônio indica a probabilidade daquela classe estar presente, independentemente das demais.	**Exemplos:** • Em uma imagem pode haver: pessoa, carro, árvore e bicicleta simultaneamente • Doenças que um paciente pode ter 

Regra prática: A configuração da camada de saída depende do tipo de problema. A quantidade de neurônios e a função de ativação são escolhidas de acordo com o formato da resposta que se deseja produzir.



HIPERPARÂMETROS
PARÂMETROS TREINÁVEIS

A quantidade de parâmetros treináveis está relacionada ao **número de camadas e ao número de neurônios por camada**.

Essa forma de cálculo é específica de camadas densas (Dense/Fully Connected), como as utilizadas em uma MLP tradicional.

Para uma camada densa:

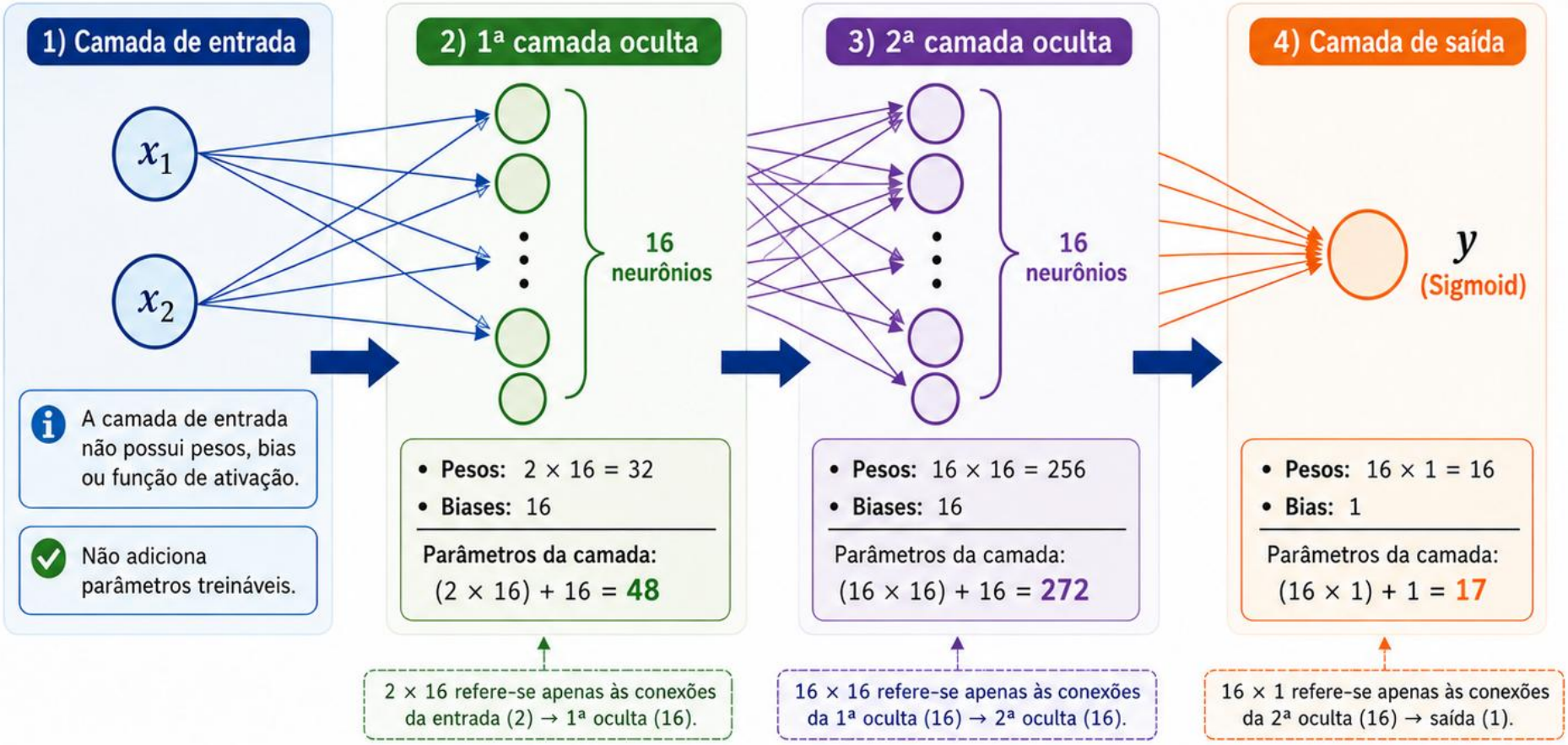
$$\text{parâmetros} = (n_{\text{entradas}} \times n_{\text{neurônios}}) + n_{\text{neurônios}}$$

Em outras arquiteturas, a lógica muda:

- **CNN:** dependem do tamanho dos filtros, canais de entrada e quantidade de filtros.
- **RNN/LSTM/GRU:** existem matrizes de pesos adicionais associadas.
- **Embedding:** dependem do tamanho do vocabulário × dimensão do embedding.
- **Camadas** de como pooling, podem não adicionar parâmetros treináveis.

Contagem de parâmetros treináveis em uma MLP

Exemplo: 2 entradas → 16 neurônios → 16 neurônios → 1 saída



Regra geral:
parâmetros de uma camada = (nº de entradas que chegam à camada × nº de neurônios da camada) + nº de biases da camada

Total de parâmetros treináveis =
48 + 272 + 17 = 337


Learning rate
Batch size
Épocas
Otimizador
Regularização
HIPERPARÂMETROS
DE TREINAMENTO
(aprendizado / otimização)



HIPERPARÂMETROS
TAMANHO DO LOTE (BATCH SIZE)

O **tamanho do lote (*batch size*)** determina **quantas instâncias do conjunto de treinamento são processadas antes de cada atualização dos pesos e biases**.

Por exemplo, para um dataset com **8.192 instâncias**, escolher *batch_size* = **128** gera **64 atualizações** dos parâmetros a cada **rodada completa sobre o dataset.**

É um **hiperparâmetro de treinamento** definido experimentalmente, pois **não existe uma regra universal** para determinar seu valor. Valores como **16, 32, 64, 128 e 256** são frequentes, mas *batches* de **512, 1024 ou maiores** também podem ser utilizados, especialmente em treinamentos de grande escala.

- **Batch grande:** observa muitas amostras para decidir → estima uma **direção mais estável** → porém precisa processar **muitas amostras antes de reavaliar a direção.**
- **Batch pequeno:** observa poucas amostras para decidir → estima uma **direção menos estável** → porém rapidamente pode **reavaliar a direção e fazer uma nova atualização.**



HIPERPARÂMETROS
NÚMERO DE ÉPOCAS (EPOCHS)

Uma **época** corresponde a **uma passagem completa de todo o conjunto de treinamento pela rede neural**. Quanto mais épocas, **mais vezes o conjunto completo de treinamento é apresentado à rede**.

Durante uma época, os dados são processados em lotes, e os parâmetros são atualizados à medida que esses lotes são processados.

Por exemplo, com **8.192 instâncias** e *batch_size* **de 128**, teremos **64 lotes por época**. Se usar como critério de parada a quantidade de **épocas igual a 32**, serão realizadas **2.048 atualizações dos parâmetros**.

Obs.: Note que **poucas épocas** podem resultar em ***underfitting*** (aprendizado insuficiente) e, por outro lado, **muitas épocas** podem resultar em ***overfitting*** (além de aumentar o tempo de treinamento e custo de processamento).



HIPERPARÂMETROS
FUNÇÃO DE PERDA (LOSS FUNCTION)

A **função de perda** (L) quantifica o quanto o modelo está distante do **objetivo definido para o treinamento**. Durante o treinamento, busca-se **minimizar essa perda** por meio do **ajuste dos pesos e biases** da rede.

Em **problemas supervisionados,** a perda é calculada comparando a **saída prevista** $\hat{y}$ com o **valor esperado** y. A forma de calcular essa perda depende do **tipo de problema**. Por exemplo:

- **Regressão**
  - **MSE (*Mean Squared Error*)** — calcula a média dos **erros ao quadrado,** penalizando mais fortemente erros maiores. Ex.: esperado = **10,** previsto = **7** → erro = $10 - 7 = 3$ → erro ao quadrado = $\mathbf{3^2 = 9}$.
  - **MAE (*Mean Absolute Error*)** — calcula a média dos **erros absolutos,** sendo menos sensível a valores extremos. Ex.: esperado = **10,** previsto = **7** → erro absoluto = $|10 - 7| = 3$.
- **Classificação Binária**
  - **Binary Cross-Entropy** – Penaliza o modelo de acordo com a **probabilidade atribuída à classe correta;** previsões confiantes e incorretas recebem perdas maiores.
- **Classificação Multiclasse**
  - **Categorical Cross-Entropy** – Penaliza o modelo de acordo com a **probabilidade atribuída à(s) classe(s) correta(s);** previsões confiantes e incorretas recebem perdas maiores.



HIPERPARÂMETROS

*GRADIENTE (*NÃO É UM HIPERPARÂMETRO EM SI)

A função de perda pode fornecer um único **valor global**, por exemplo $L = 2$.

Porém, esse valor não indica **como ajustar cada peso e bias da rede**.

O gradiente resolve isso calculando, para **cada parâmetro**, sua influência sobre a perda. Assim, cada parâmetro pode ter **direção e intensidade de influência diferentes**; o sinal do gradiente indica a direção local de aumento da perda e, para minimizá-la, o Gradient Descent atua na direção oposta.

$$L \xrightarrow{\text{backpropagation}} \frac{\partial L}{\partial w_i}, \frac{\partial L}{\partial b_i}$$

Assim, para **L = 2**, podemos ter, por exemplo, algo como:
peso $w_1 \rightarrow$ gradiente $+\mathbf{0,8}$
peso $w_2 \rightarrow$ gradiente $-\mathbf{0,3}$
bias $b_1 \rightarrow$ gradiente $+\mathbf{0,05}$
peso $w_3 \rightarrow$ gradiente $+\mathbf{0,01}$
bias $b_2 \rightarrow$ gradiente $\mathbf{-0,4}$
...



HIPERPARÂMETROS
TAXA DE APRENDIZADO

A **taxa de aprendizado** (*learning rate*) determina a **magnitude das alterações realizadas nos pesos e biases** a cada atualização durante o treinamento. Normalmente é representada por **η** (*eta*) ou **α**.

A partir da **função de perda,** são calculados os **gradientes**, que indicam a direção em que cada parâmetro deve ser alterado para reduzir o erro. A taxa de aprendizado controla o **tamanho do passo** realizado nessa direção (ou seja, **com qual "agressividade" os gradientes serão aplicados**).

Valores típicos de *learning rate* são **0.01, 0.001 e 0.0001**.

Obs.: taxas muito altas produzem **passos grandes,** podendo ultrapassar regiões de mínimo e dificultar a convergência. Taxas muito baixas produzem **passos pequenos,** tornando o aprendizado mais lento.

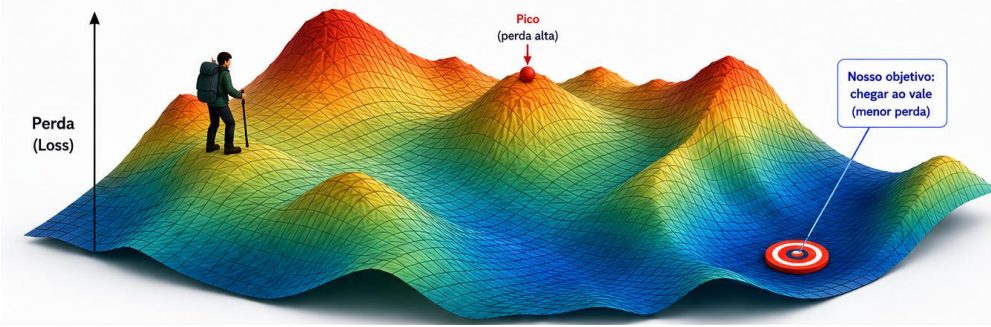


HIPERPARÂMETROS
OTIMIZADOR

A partir do momento em que os gradientes foram calculados, já sabemos como cada peso e *bias* influencia a função de perda. O próximo passo é **utilizar essa informação para atualizar os parâmetros**.

O **otimizador** é o **algoritmo responsável** por determinar **como essa atualização será realizada**, utilizando os gradientes calculados na retropropagação (*backpropagation*).

O **Gradiente Descendente (*Gradient Descent*)** representa a ideia fundamental desse processo. Diferentes otimizadores utilizam os gradientes de maneiras distintas, buscando melhorar a **velocidade e a estabilidade da convergência**.





HIPERPARÂMETROS
OTIMIZADOR

Entre os principais estão:

- **SGD (*Stochastic Gradient Descent*)** — no **SGD puro**, a regra é: **"pegue o gradiente, multiplique pela taxa de aprendizado e altere o parâmetro na direção oposta."**

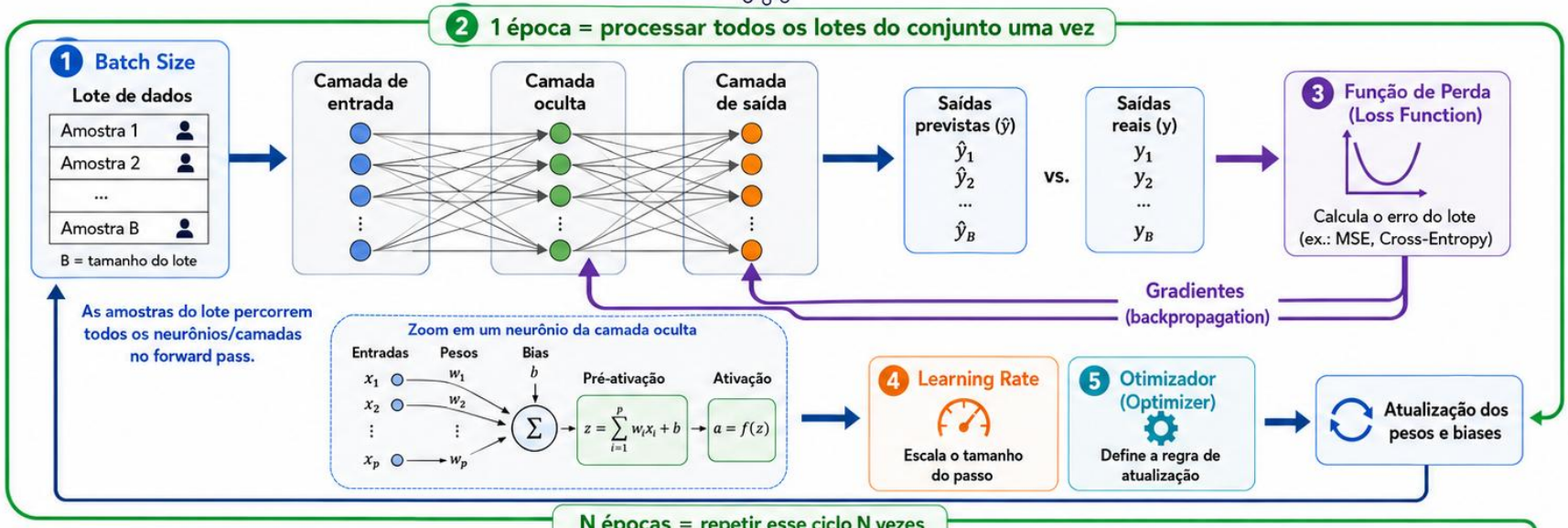
$$w_{novo} = w_{atual} - \eta \cdot g$$
$$w_{novo} = w_{atual} - (0{,}01 \times 0{,}8)$$
$$w_{novo} = w_{atual} - 0{,}008$$

Outros Otimizadores:

- **SGD com Momentum** — acrescenta ao SGD uma espécie de **"inércia"**, considerando também atualizações anteriores para estabilizar e reduzi oscilações.
- **RMSprop — adapta o tamanho das atualizações para cada parâmetro** com base no histórico recente dos gradientes, reduzindo passos em parâmetros que apresentam gradientes persistentemente maiores.
- **Adam** — combina ideias de **Momentum e RMSprop**, considerando tanto a direção acumulada dos gradientes quanto suas magnitudes recentes. É um dos otimizadores mais utilizados principalmente em redes profundas.
- **AdamW** — variante do Adam que trata de forma separada a **regularização dos pesos (*weight decay*)**, sendo bastante utilizado em modelos modernos.

HIPERPARÂMETROS DE TREINAMENTO EM REDES NEURAIS

Onde cada hiperparâmetro atua durante o treinamento



1 Tamanho do Lote (Batch Size)

- Quantas amostras formam cada lote.
- Atua na entrada de cada ciclo de treinamento.

2 Número de Épocas (Epochs)

- Quantas vezes todos os lotes são processados.
- Atua no ciclo global de treinamento.

3 Função de Perda (Loss Function)

- Mede o erro entre ŷ e y.
- Atua após a camada de saida.

4 Taxa de Aprendizado (Learning Rate)

- Controla a magnitude da atualização.
- Atua na etapa de atualização dos parâmetros.

5 Otimizador (Optimizer)

- Transforma gradientes em atualização dos parâmetros.
- Atua junto com o learning rate.

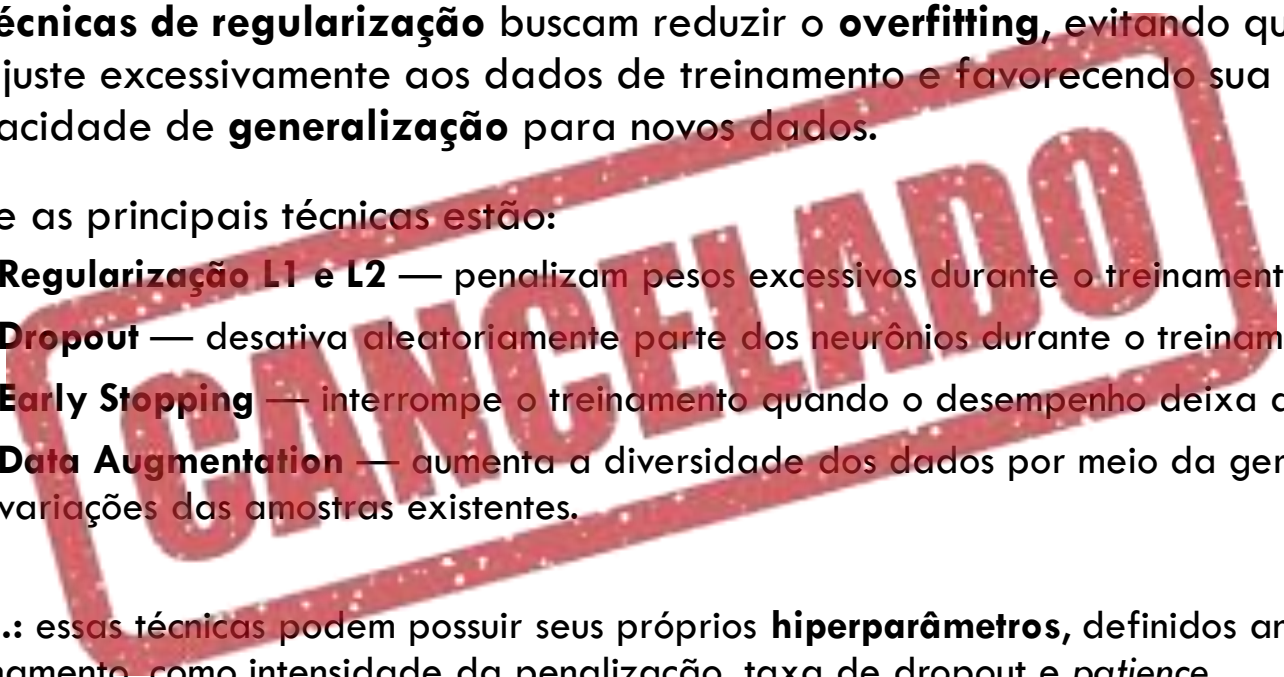
Fluxo do treinamento: lote → forward pass (rede) → saída (ŷ) → loss → gradientes (backprop) → otimizador + learning rate → atualização (parâmetros)

TÉCNICAS DE REGULARIZAÇÃO

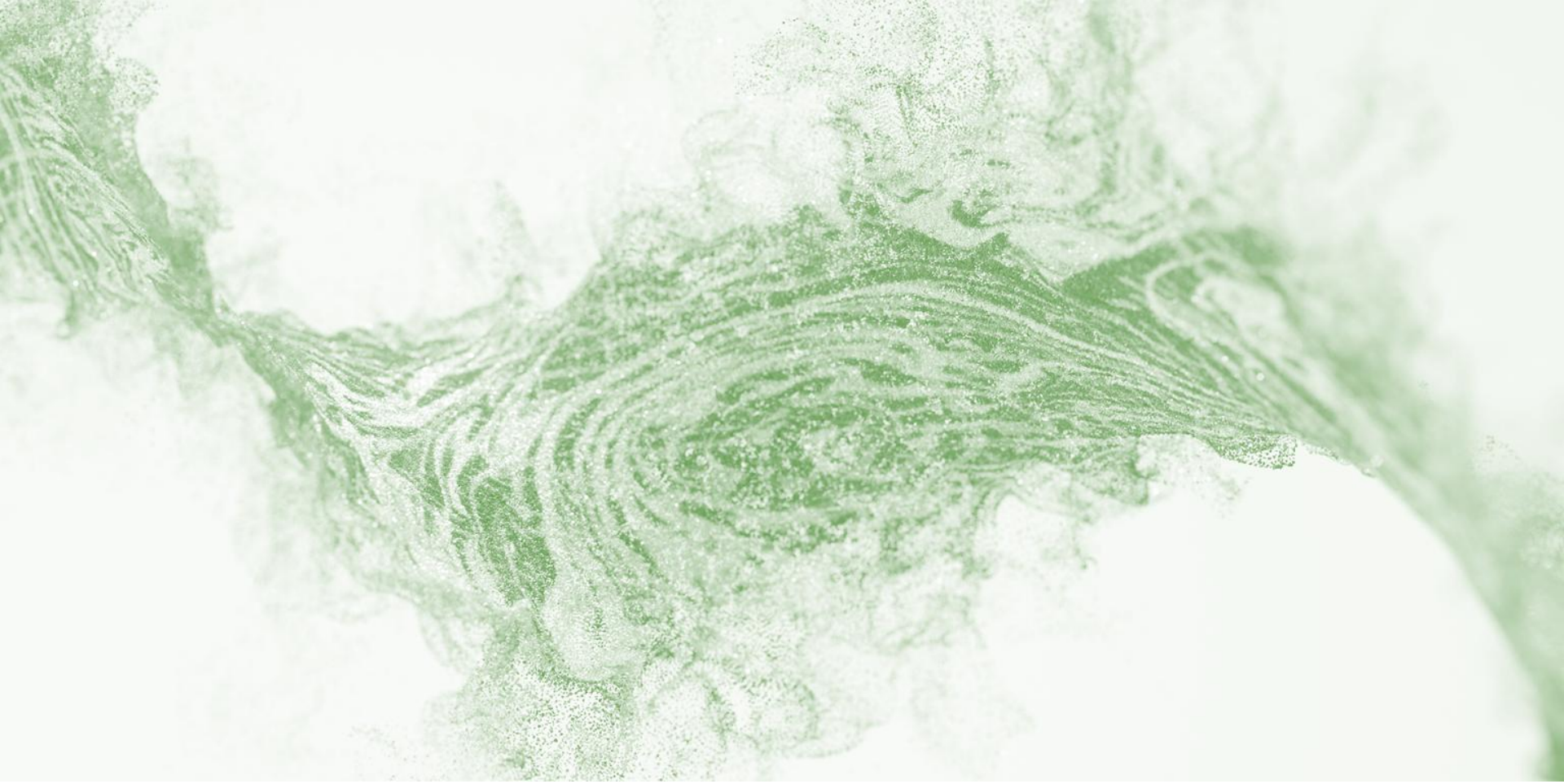
As **técnicas de regularização** buscam reduzir o **overfitting**, evitando que a rede se ajuste excessivamente aos dados de treinamento e favorecendo sua capacidade de **generalização** para novos dados.

Entre as principais técnicas estão:

- **Regularização L1 e L2** — penalizam pesos excessivos durante o treinamento.
- **Dropout** — desativa aleatoriamente parte dos neurônios durante o treinamento.
- **Early Stopping** — interrompe o treinamento quando o desempenho deixa de melhorar.
- **Data Augmentation** — aumenta a diversidade dos dados por meio da geração de variações das amostras existentes.



Obs.: essas técnicas podem possuir seus próprios **hiperparâmetros**, definidos antes do treinamento, como intensidade da penalização, taxa de dropout e *patience*.



III – ARQUITETURAS DE REDES NEURAIS

Prof. Luis Claudio
M.Sc. Eng. Elétrica(PUC-Rio-06)
Eng. de Computação (ITA-99)

ARQUITETURAS DE REDES NEURAIS

Até agora, estudamos principalmente **Redes Neurais do tipo MLP (*Multilayer Perceptron*)**, utilizadas como base para compreender neurônios, camadas, funções de ativação, parâmetros e treinamento.

Existem outras arquiteturas de redes neurais, entre elas:

- **CNN (*Convolutional Neural Networks*)** - Especialmente adequadas para imagens e dados com estrutura espacial. Utilizam operações de convolução para identificar padrões locais, como **bordas, formas** e outras **estruturas visuais**.
- **RNN (*Recurrent Neural Networks*)** - Voltadas ao processamento de **sequências e dados temporais**. Mantêm informações de estados anteriores, permitindo considerar a ordem e a dependência entre elementos da sequência.
- **LSTM (*Long Short-Term Memory*) e GRU (*Gated Recurrent Unit*)** - Variações de RNN capazes de lidar melhor com dependências de longo prazo. Utilizam mecanismos de controle do fluxo de informação para reduzir problemas como o vanishing gradient.
- **Transformers** - Baseados em **mecanismos de atenção**, especialmente self-attention, permitindo modelar relações entre diferentes posições de uma sequência.



CNN
CONVOLUTIONAL NEURAL NETWORKS

As ideias que levaram às CNNs remontam ao **Neocognitron**, proposto por **Kunihiko Fukushima em 1980**.

Posteriormente, **Yann LeCun e colaboradores** consolidaram o uso de redes convolucionais treinadas por *backpropagation*, destacando-se o **LeNet-5, em 1998**, aplicado ao **reconhecimento de caracteres manuscritos**.

Durante muitos anos, as CNNs foram a arquitetura dominante em **Visão Computacional**. A partir dos **Vision Transformers (ViTs)**, apresentados em 2020, Transformers passaram a competir e, em diversas tarefas, superar arquiteturas puramente convolucionais.

Ainda hoje, revisões recentes continuam tratando CNNs como arquiteturas relevantes tanto em aplicações acadêmicas quanto industriais, inclusive em classificação, imagens médicas, sistemas autônomos e cenários com restrição computacional de processamento.



CNN
CONVOLUTIONAL NEURAL NETWORKS

As **Redes Neurais Convolucionais (CNNs)** são redes neurais desenvolvidas especialmente para dados com **estrutura espacial**, principalmente **imagens**.

Diferentemente de uma MLP, em que cada neurônio de uma camada densa se conecta a todas as saídas da camada anterior, as CNNs utilizam **filtros** (***kernels***) que percorrem **pequenas regiões da entrada** realizando **convoluções**.

Esses filtros permitem aprender **padrões locais**, como:

- **bordas → texturas → formas → estruturas mais complexas**

CNN
CONVOLUTIONAL NEURAL NETWORKS

Como uma imagem é representada para uma rede neural?

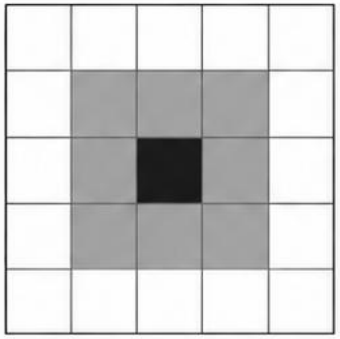
Uma imagem digital é formada por uma matriz de **pixels**, e cada pixel é representado numericamente.

- **Escala de Cinza** - Em imagens em **escala de cinza**, cada pixel possui um único valor de intensidade, normalmente entre **0 e 255**. Assim, uma imagem de **28 × 28 pixels** pode ser representada por **784 valores**. No laboratório com Fashion-MNIST, esses 784 valores foram utilizados como entradas da MLP.
- **Escala RGB** - Em imagens coloridas no padrão **RGB**, cada pixel é representado por **três valores**: **R (Red), G (Green) e B (Blue)**, também geralmente variando de **0 a 255**.
- **Escala RGBA** - O padrão **RGBA** acrescenta um quarto valor ao RGB: **A (Alpha)**, responsável por representar o nível de **transparência ou opacidade** do pixel. Assim, cada pixel é representado por **quatro valores: R, G, B e A**.

Como uma imagem é representada para uma rede neural

Imagem em escala de cinza

Imagem (5 × 5 pixels)



Representação numérica
(matriz de intensidades)

$$\begin{bmatrix} 255 & 255 & 255 & 255 & 255 \\ 255 & 180 & 180 & 180 & 255 \\ 255 & 180 & 0 & 180 & 255 \\ 255 & 180 & 180 & 180 & 255 \\ 255 & 255 & 255 & 255 & 255 \end{bmatrix}$$

1 valor por pixel

valores de intensidade
(0 a 255)
(0 = preto, 255 = branco)

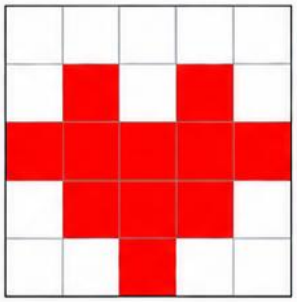
Forma da imagem: H × W × 1



Ex.: pixel = **180**
(tom de cinza intermediário)

Imagem colorida (RGB)

Imagem (5 × 5 pixels)



Representação numérica
(3 canais: **R**, G, B)

R (Red)

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 255 & 0 & 255 & 0 \\ 255 & 255 & 255 & 255 & 255 \\ 0 & 255 & 255 & 255 & 0 \\ 0 & 0 & 255 & 0 & 0 \end{bmatrix}$$

G (Green)

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

B (Blue)

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

3 valores por pixel

R (Red), **G** (Green), **B** (Blue)
valores de intensidade (0 a 255)

Forma da imagem: H × W × 3



Ex.: pixel = (**255,** 0, 0)
(pixel vermelho)

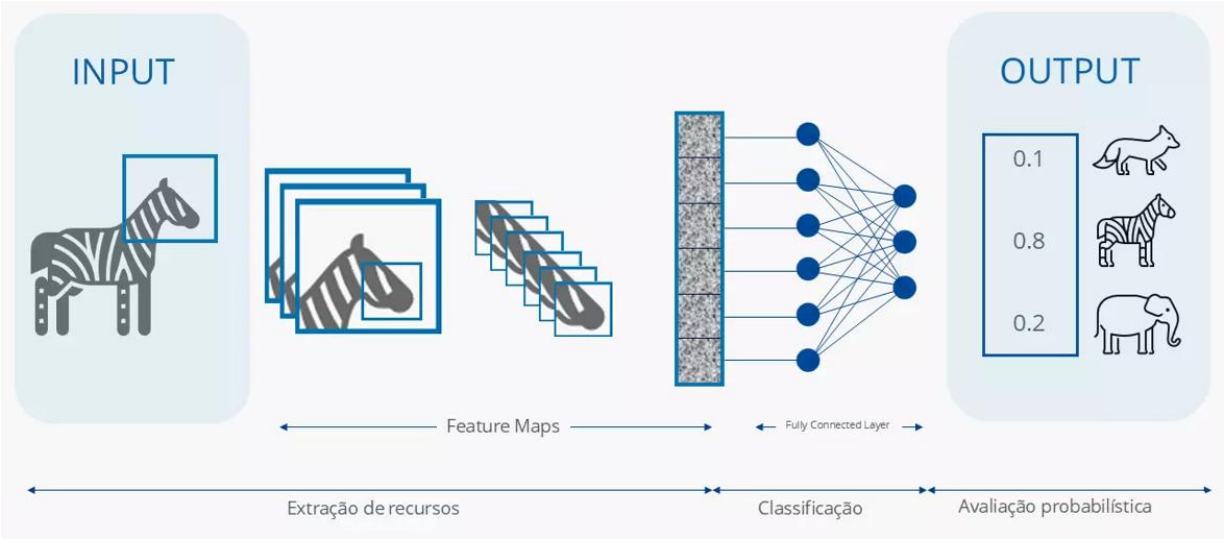
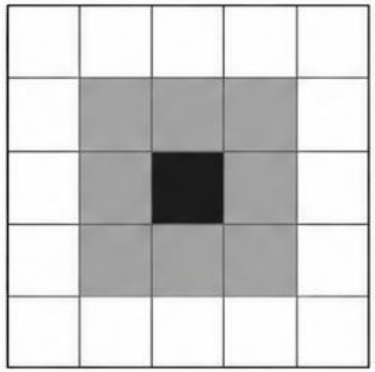


Imagem (5 × 5 pixels)

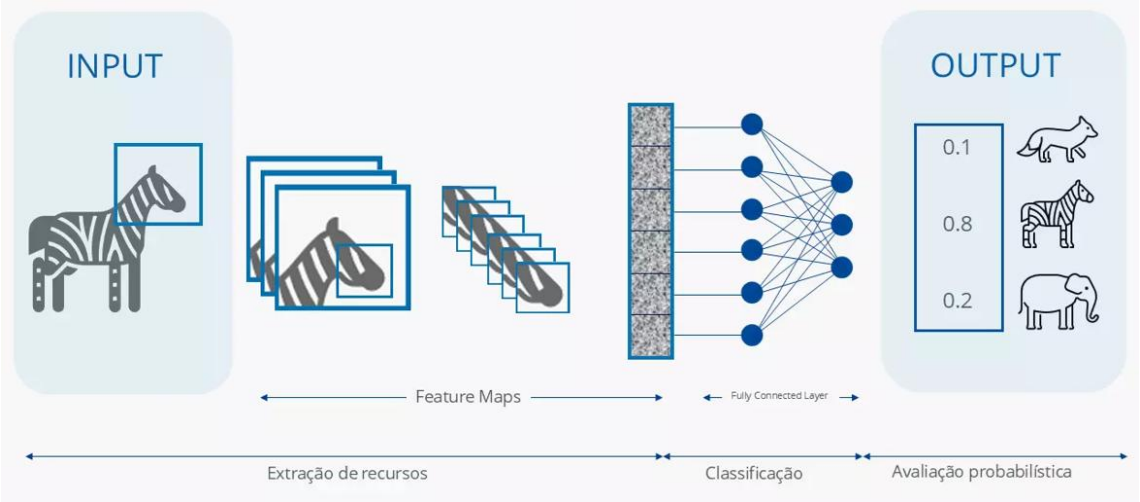


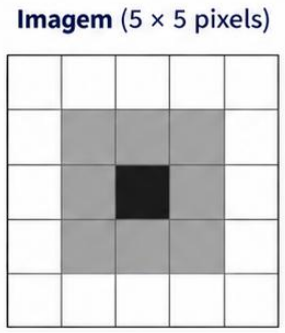
Representação numérica
(matriz de intensidades)

$$\begin{bmatrix} 255 & 255 & 255 & 255 & 255 \\ 255 & 180 & 180 & 180 & 255 \\ 255 & 180 & 0 & 180 & 255 \\ 255 & 180 & 180 & 180 & 255 \\ 255 & 255 & 255 & 255 & 255 \end{bmatrix}$$

CNN
ENTRADA DA CNN

Uma imagem em escala de cinza pode ser **representada por uma matriz**, em que cada posição corresponde à intensidade de um pixel. Na CNN, essa matriz é mantida como estrutura espacial, permitindo que **regiões vizinhas da imagem sejam analisadas em conjunto.**





Representação numérica (matriz de intensidades)

255	255	255	255	255
255	180	180	180	255
255	180	0	180	255
255	180	180	180	255
255	255	255	255	255

CNN
FILTRO (KERNEL)

Um filtro é uma pequena matriz de **pesos treináveis** aplicada sobre regiões locais da imagem. O **filtro percorre a imagem** analisando pequenas regiões, usando sempre os mesmos pesos.

Essa é justamente a operação de **convolução**: uma matriz pequena percorrendo uma matriz maior.

A varredura acontece:

- da **esquerda para a direita,** conforme o stride horizontal;
- de **cima para baixo,** conforme o stride vertical.

Exemplo de filtro 3×3:

$$K = \begin{bmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{bmatrix} \qquad X = \begin{bmatrix} 255 & 255 & 255 & 255 & 255 \\ 255 & 180 & 180 & 180 & 255 \\ 255 & 180 & 0 & 180 & 255 \\ 255 & 180 & 180 & 180 & 255 \\ 255 & 255 & 255 & 255 & 255 \end{bmatrix}$$

CNN
FILTRO (KERNEL)

Aplicando o filtro sobre a imagem, essa operação produz **um único valor** da nova matriz de características para cada posição de deslocamento do filtro. **Cada posição do filtro produz um valor da matriz de características**.

1ª posição:

$$\begin{bmatrix} 255 & 255 & 255 \\ 255 & 180 & 180 \\ 255 & 180 & 0 \end{bmatrix} \qquad X = \begin{bmatrix} 255 & 255 & 255 & 255 & 255 \\ 255 & 180 & 180 & 180 & 255 \\ 255 & 180 & 0 & 180 & 255 \\ 255 & 180 & 180 & 180 & 255 \\ 255 & 255 & 255 & 255 & 255 \end{bmatrix}$$

$$z_{11} = 255w_1 + 255w_2 + 255w_3 + 255w_4 + 180w_5 + 180w_6 + 255w_7 + 180w_8 + 0w_9 + b$$

2ª posição:

$$\begin{bmatrix} 255 & 255 & 255 \\ 180 & 180 & 180 \\ 180 & 0 & 180 \end{bmatrix} \qquad X = \begin{bmatrix} 255 & 255 & 255 & 255 & 255 \\ 255 & 180 & 180 & 180 & 255 \\ 255 & 180 & 0 & 180 & 255 \\ 255 & 180 & 180 & 180 & 255 \\ 255 & 255 & 255 & 255 & 255 \end{bmatrix}$$

$$z_{12} = 255w_1 + 255w_2 + 255w_3 + 180w_4 + 180w_5 + 180w_6 + 180w_7 + 0w_8 + 180w_9 + b$$

Demais posições: $z_{13},\ z_{21},\ z_{22}, \ldots$

CNN
MAPA DE CARACTERÍSTICAS (A = F(Z))

À medida que o filtro percorre a imagem, **cada posição gera um valor**. Esses valores são organizados em uma nova matriz Z (matriz de pré-ativação).

Exemplo de filtro 3×3:

$$K = \begin{bmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{bmatrix} \quad X = \begin{bmatrix} 255 & 255 & 255 & 255 & 255 \\ 255 & 180 & 180 & 180 & 255 \\ 255 & 180 & 0 & 180 & 255 \\ 255 & 180 & 180 & 180 & 255 \\ 255 & 255 & 255 & 255 & 255 \end{bmatrix} \quad Z = \begin{bmatrix} z_{11} & z_{12} & z_{13} \\ z_{21} & z_{22} & z_{23} \\ z_{31} & z_{32} & z_{33} \end{bmatrix}$$

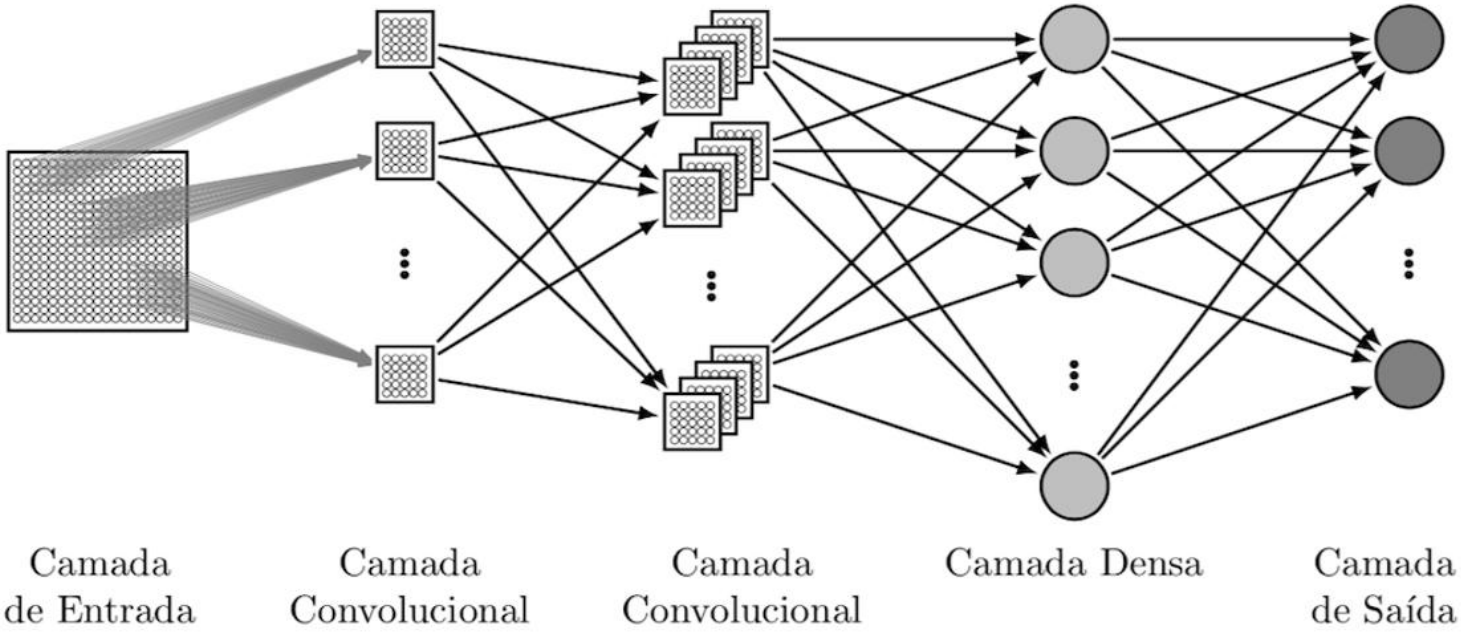
A matriz A é o **mapa de características** (***feature map***) gerado pelo filtro.

$$A = \mathrm{ReLU}(Z) \qquad A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

$$\boxed{X_{5\times5}} \rightarrow \boxed{K_{3\times3}} \rightarrow \boxed{Z = \begin{bmatrix} z_{11} & z_{12} & z_{13} \\ z_{21} & z_{22} & z_{23} \\ z_{31} & z_{32} & z_{33} \end{bmatrix}} \rightarrow \boxed{A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}}$$

CNN
PRIMEIRA CAMADA DE CONVOLUÇÃO

Assim como na MLP, cada camada oculta pode ter vários neurônios, **na CNN cada camada convolucional pode ter vários filtros**. Os filtros de uma mesma camada convolucional recebem a mesma entrada e operam sobre ela em paralelo.



CNN
PRIMEIRA CAMADA DE CONVOLUÇÃO

Por isso, em cada camada convolucional de uma CNN, geralmente definimos o a **quantidade de filtros** e também o **tamanho do kernel**.

```python
Conv2D(filters=2, kernel_size=(3,3))
```

Cada filtro possui pesos diferentes e gera **seu próprio mapa** de características.

$$X \rightarrow \begin{cases} K_1 \rightarrow Z_1 \rightarrow A_1 \\ K_2 \rightarrow Z_2 \rightarrow A_2 \end{cases}$$

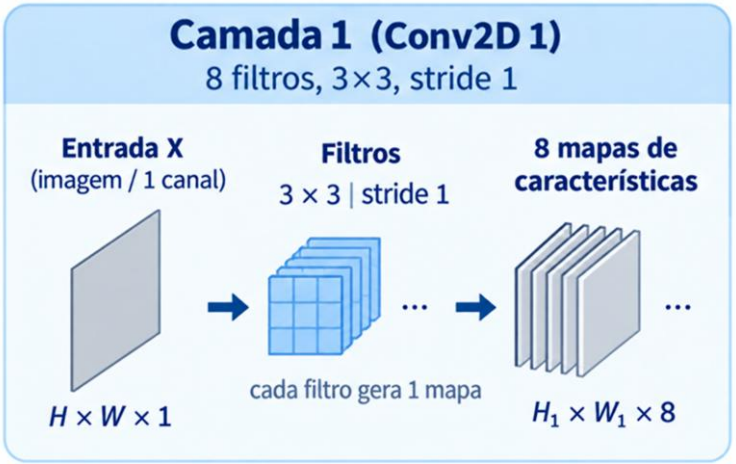
$$A_1 = \begin{bmatrix} a_{11}^{(1)} & a_{12}^{(1)} & a_{13}^{(1)} \\ a_{21}^{(1)} & a_{22}^{(1)} & a_{23}^{(1)} \\ a_{31}^{(1)} & a_{32}^{(1)} & a_{33}^{(1)} \end{bmatrix}$$

$$A_2 = \begin{bmatrix} a_{11}^{(2)} & a_{12}^{(2)} & a_{13}^{(2)} \\ a_{21}^{(2)} & a_{22}^{(2)} & a_{23}^{(2)} \\ a_{31}^{(2)} & a_{32}^{(2)} & a_{33}^{(2)} \end{bmatrix}$$

CNN
PRIMEIRA CAMADA DE CONVOLUÇÃO

Quando a entrada possui apenas um canal, **cada** filtro é formado por **uma única matriz** (tem "profundidade 1"). Em uma imagem em **escala de cinza**, temos apenas 1 canal de entrada. Então um filtro 3x3 será **uma única matriz 3x3**.

$$X * K$$



Como existe apenas um canal, **não é necessário combinar resultados de diferentes canais.** O resultado da convolução, somado ao bias, gera diretamente:

$$Z = X * K + b$$

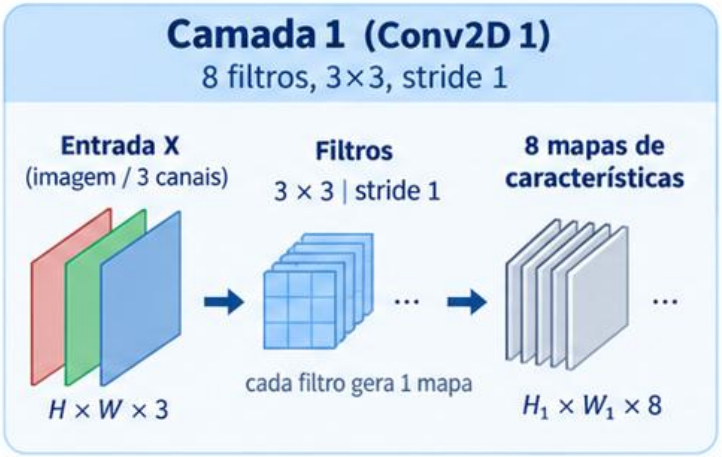
CNN
PRIMEIRA CAMADA DE CONVOLUÇÃO

Quando a entrada possui apenas **vários canais**, **cada** filtro é formado por **uma matriz para cada canal** (tem "profundidade N"). Em uma imagem **RGB**, temos 3 canais de entrada. Então um filtro 3x3 será **3 matrizes 3x3**.

$$X_R * K_R$$

$$X_G * K_G$$

$$X_B * K_B$$



Como existem vários canais, **os resultados precisam ser combinados.** Eles são somados, juntamente com o bias:

$$Z = X_R * K_R + X_G * K_G + X_B * K_B + b$$

CNN
N-ÉSIMA CAMADA DE CONVOLUÇÃO

A próxima camada convolucional recebe como entrada **todos os mapas de características gerados pela camada anterior**. Se a camada anterior gerou 2 mapas, A1 e A2, a próxima camada recebe uma entrada com **profundidade 2.**

Consequentemente, **cada filtro da próxima camada terá uma matriz de pesos para A1 e outra para A2.**

Os resultados são combinados para gerar um único mapa Z por filtro:

$$Z = A_1 * K_1 + A_2 * K_2 + b$$

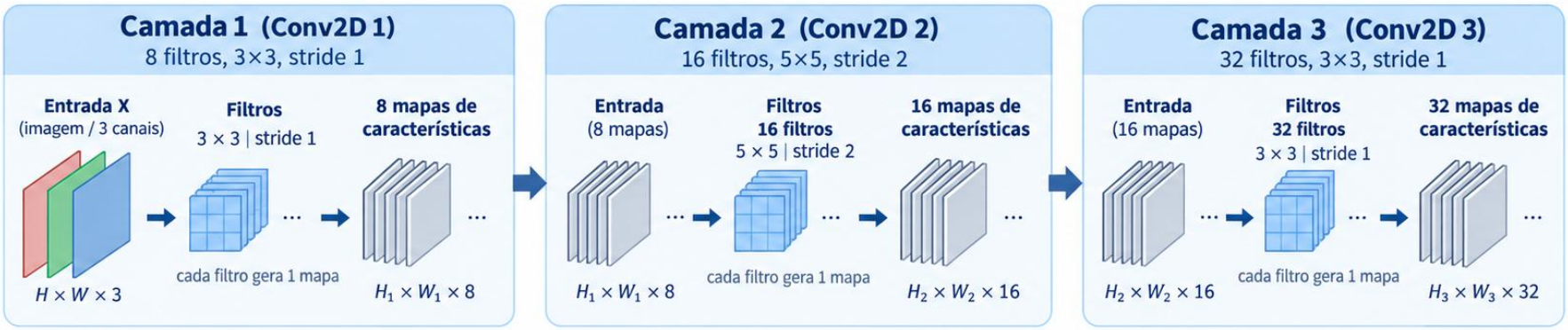
Em outras palavras, **cada mapa de características produzido por uma camada passa a funcionar como um canal de entrada para a próxima camada.**

CNN
N-ÉSIMA CAMADA DE CONVOLUÇÃO

Cada filtro atua sobre todos os canais da entrada. Como todos os filtros da camada recebem a mesma entrada, esses cálculos podem ocorrer em paralelo.

Se a entrada tem $\mathbf{C}$ canais e a camada possui $\boldsymbol{F}$ filtros, cada um destes $\boldsymbol{F}$ filtros possui profundidade $\mathbf{C}$ (várias matrizes) e gera $\mathbf{UMA}$ matriz de características.

```python
Conv2D(filters=8,  kernel_size=(3,3), strides=(1,1), activation='relu')
Conv2D(filters=16, kernel_size=(5,5), strides=(2,2), activation='relu')
Conv2D(filters=32, kernel_size=(3,3), strides=(1,1), activation='relu')
```



CNN
FILTRO VERSUS CANAIS DE ENTRADA

Os filtros consideram **simultaneamente todos os feature maps** da camada anterior e combinam os valores correspondentes para produzir **um único valor z**.

$$K^{(1)} = \begin{bmatrix} w_1^{(1)} & w_2^{(1)} \\ w_3^{(1)} & w_4^{(1)} \end{bmatrix}$$

$$K^{(2)} = \begin{bmatrix} w_1^{(2)} & w_2^{(2)} \\ w_3^{(2)} & w_4^{(2)} \end{bmatrix}$$

Na primeira posição, ele pega a região 2×2 de A_1:

$$\begin{bmatrix} a_{11}^{(1)} & a_{12}^{(1)} \\ a_{21}^{(1)} & a_{22}^{(1)} \end{bmatrix}$$

e também a região correspondente de A_2:

$$\begin{bmatrix} a_{11}^{(2)} & a_{12}^{(2)} \\ a_{21}^{(2)} & a_{22}^{(2)} \end{bmatrix}$$

O novo valor é calculado combinando **os dois mapas**:

$$z_{11} = a_{11}^{(1)} w_1^{(1)} + a_{12}^{(1)} w_2^{(1)} + a_{21}^{(1)} w_3^{(1)} + a_{22}^{(1)} w_4^{(1)}$$

$$+ a_{11}^{(2)} w_1^{(2)} + a_{12}^{(2)} w_2^{(2)} + a_{21}^{(2)} w_3^{(2)} + a_{22}^{(2)} w_4^{(2)} + b$$

CNN
PADDING E STRIDE

Stride - Define **quantas posições o filtro avança** a cada movimento. Quanto maior o stride, **menor tende a ser a saída.**

- **stride = 1:** o filtro avança **1 posição por vez.**
- **stride = 2:** o filtro avança **2 posições por vez.**

Padding - Adiciona uma **borda**, normalmente com zeros, ao redor da matriz de entrada. Serve para:

- **Evitar que a imagem diminua** muito após cada convolução;
- Permitir que o filtro processe melhor os **pixels das bordas.**

```python
Conv2D(filters=2, kernel_size=(3,3), strides=(1,1), padding='valid')
```

Stride e Padding

Dois conceitos importantes em Convolutional Neural Networks (CNNs)

Stride

Stride = passo do filtro.

Entrada (5 × 5)

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

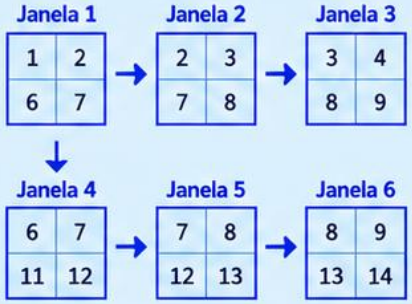
Filtro / Kernel (2 × 2)

1	0
0	-1

O kernel (2 × 2) desliza sobre a entrada, avançando de acordo com o stride.

Stride = 1

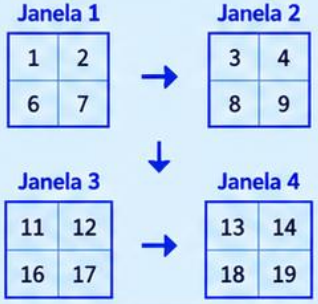
O filtro avança 1 posição por vez e depois desce para a próxima linha.



Há **sobreposição** entre janelas adjacentes.

Stride = 2

O filtro avança 2 posições por vez e depois desce 2 linhas.



Aqui **não há sobreposição** entre janelas.

Quanto maior o stride, menor tende a ser a saída.

Padding

Padding = borda adicionada à entrada.

Sem padding (valid)

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

Sem padding, o kernel só pode atuar nas posições em que ele cabe completamente dentro da entrada, não alcançando as bordas.

Com padding = 1 (same)

0	0	0	0	0	0	0
0	1	2	3	4	5	0
0	6	7	8	9	10	0
0	11	12	13	14	15	0
0	16	17	18	19	20	0
0	21	22	23	24	25	0
0	0	0	0	0	0	0

Com padding = 1, adicionamos uma **borda** de zeros. Assim, o kernel também consegue atuar nas bordas da entrada.

Exemplo: o kernel atuando nas bordas

0	0	0	0	0	0	0
0	1	2	3	4	5	0
0	6	7	8	9	10	0
0	11	12	13	14	15	0
0	16	17	18	19	20	0
0	21	22	23	24	25	0
0	0	0	0	0	0	0

Kernel atuando no canto superior esquerdo (graças ao padding).

0	0	0	0	0	0	0
0	1	2	3	4	5	0
0	6	7	8	9	10	0
0	11	12	13	14	15	0
0	16	17	18	19	20	0
0	21	22	23	24	25	0
0	0	0	0	0	0	0

Kernel atuando no canto inferior direito (graças ao padding).

O padding ajuda a preservar o tamanho e tratar melhor as bordas.

Stride controla o passo do filtro. Padding controla a borda da entrada.

CNN
POOLING

O **Pooling** é uma operação aplicada sobre os mapas de características para **reduzir suas dimensões espaciais**. No **Max Pooling**, uma pequena janela percorre cada mapa e mantém apenas o maior valor encontrado em cada região.

Se você quiser aplicar **pooling após cada camada convolucional**, ficaria assim:

CANCELADO

Python

Run

```
Conv2D(filters=8, kernel_size=(3,3), activation='relu')
MaxPooling2D(pool_size=(2,2))

Conv2D(filters=16, kernel_size=(3,3), activation='relu')
MaxPooling2D(pool_size=(2,2))

Conv2D(filters=32, kernel_size=(3,3), activation='relu')
MaxPooling2D(pool_size=(2,2))
```

CNN
CAMADAS DENSAS

Depois de várias camadas convolucionais, a rede possui um conjunto de **mapas de características** que representam os padrões aprendidos da imagem.

Tipicamente, esses mapas são então transformados em uma representação adequada para uma **camada densa**, que realizará a etapa final de classificação. Uma forma clássica é utilizar o **Flatten**, que transforma todos os valores dos mapas de características **em um único vetor**.

Ou seja, **4 feature maps**, cada um 2×2:

$$A_1 = \begin{bmatrix} a_1 & a_2 \\ a_3 & a_4 \end{bmatrix} \quad A_2 = \begin{bmatrix} a_5 & a_6 \\ a_7 & a_8 \end{bmatrix} \quad A_3 = \begin{bmatrix} a_9 & a_{10} \\ a_{11} & a_{12} \end{bmatrix} \quad A_4 = \begin{bmatrix} a_{13} & a_{14} \\ a_{15} & a_{16} \end{bmatrix}$$

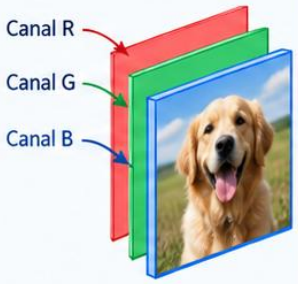
O `Flatten` transforma tudo em: $\boxed{[a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8, \ldots, a_{16}]}$

CNN → MLP: das camadas convolucionais às camadas densas

As camadas convolucionais extraem padrões da imagem e geram mapas de características, que são transformados em um vetor (Flatten) para serem processados por camadas densas (MLP).

1. Entrada

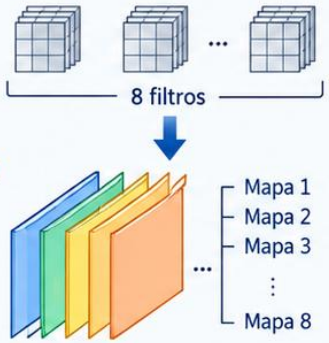
Imagem RGB
(com 3 canais)



Entrada $X = H \times W \times 3$

2. Camada convolucional 1

8 filtros 3 × 3

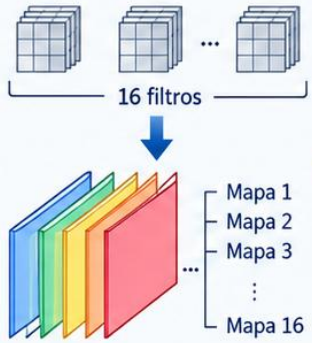


Saída: $H_1 \times W_1 \times 8$

Cada filtro gera 1 mapa de características.

3. Camada convolucional 2

16 filtros 3 × 3

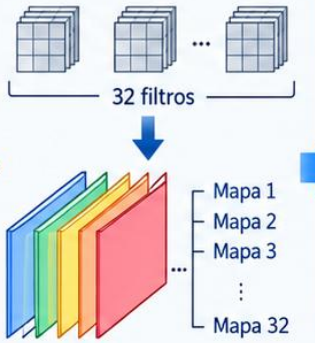


Saída: $H_2 \times W_2 \times 16$

Número de mapas de saída = número de filtros.

4. Camada convolucional 3

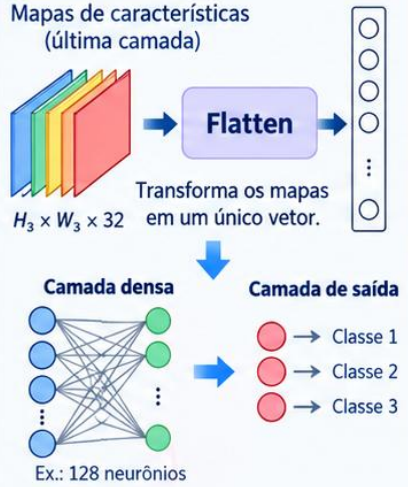
32 filtros 3 × 3



Saída: $H_3 \times W_3 \times 32$

Cada mapa de saída corresponde a um canal.

5. Flatten → MLP (classificação)



MLP / classificação

Resumo:

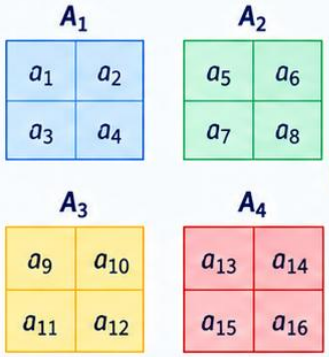
1. As camadas convolucionais **extraem características.**
2. Os mapas de características tornam-se os **canais da próxima camada.**
3. Após o Flatten, o vetor alimenta as **camadas densas.**

CNN → MLP: do Flatten às camadas densas

Após a última camada convolucional, os mapas de características são reorganizados em um vetor pelo Flatten e, em seguida, processados pelas camadas densas para realizar a classificação.

1. Saída convolucional

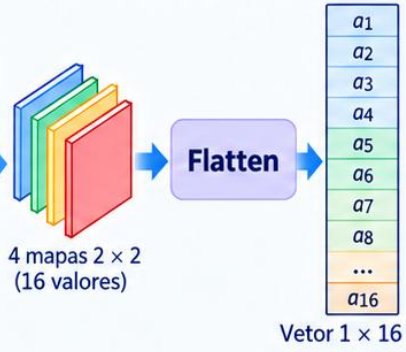
4 mapas de características (2 × 2 cada)



Ex.: 4 mapas 2 × 2

2. Flatten

Reorganiza todos os valores dos mapas em um único vetor (sem descartar informações).



Flatten apenas reorganiza os valores em um único vetor.
Não aprende parâmetros.

3. Vetor de entrada da MLP

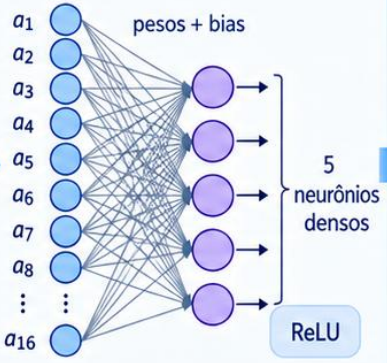
Vetor de entrada (16 valores)



Todos os valores dos mapas tornam-se entradas da camada densa.

4. Camada densa

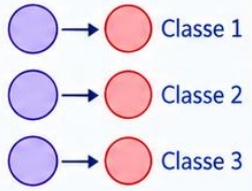
Cada neurônio combina todas as entradas do vetor.



Agora há processamento com parâmetros aprendidos.

5. Saída

Camada de saída (3 classes)



A MLP usa o vetor para realizar a classificação.

Resumo:

1. Os mapas de características são transformados **em um vetor.**
2. O Flatten só reorganiza os dados. **Não aprende parâmetros.**
3. As **camadas densas** recebem o vetor e fazem a classificação.

MLP → CNN: o que permanece e o que surge

Resumo didático dos parâmetros treináveis e dos hiperparâmetros ao passar de MLP para CNN.

Categoria	Em MLP	Em CNN
Parâmetros treináveis	Pesos das camadas densas Biases das camadas densas	Pesos das camadas densas Biases das camadas densas Pesos dos filtros convolucionais Biases dos filtros convolucionais
Hiperparâmetros de arquitetura que permanecem	Nº de camadas densas Neurônios por camada Funções de ativação	Esses elementos podem continuar na parte densa final
Hiperparâmetros de treinamento que permanecem	Batch size Epochs Loss function Learning rate Optimizer	Permanecem essencialmente os mesmos
Hiperparâmetros que surgem em CNN	Não se aplicam	**Nº de camadas convolucionais** **Nº de filtros** **Tamanho do kernel** **Stride** **Padding** **Pooling***

* **Pooling**: ao utilizá-lo, surgem escolhas específicas, como tipo de pooling, pool size e stride do pooling.

Ideia central: em CNN, pesos e biases permanecem, mas surgem filtros convolucionais e novos hiperparâmetros específicos.

ARQUITETURAS DE REDES NEURAIS
RNN - RECURRENT NEURAL NETWORKS

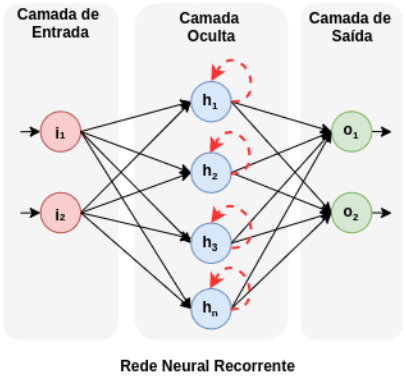
Redes tradicionais, como MLPs, processam cada entrada **sem manter informação sobre o que foi processado anteriormente**.

Por exemplo, em PLN, representações como **BoW** e **TF-IDF** transformam um documento inteiro em um vetor, perdendo explicitamente a **ordem das palavras**.

Isso é um problema quando o significado depende da sequência:

"cachorro morde homem" ≠ "homem morde cachorro"

A partir das décadas de **1980 e 1990**, arquiteturas recorrentes, como as propostas por **Jordan e Elman**, passaram a **explorar conexões capazes de manter informação de estados anteriores**.



Rede Neural Recorrente

ARQUITETURAS DE REDES NEURAIS
RNN - RECURRENT NEURAL NETWORKS

As Redes Neurais Recorrentes (**RNNs)** são redes neurais desenvolvidas para trabalhar com **dados sequenciais,** nos quais a **ordem dos elementos** é importante, como **séries temporais, textos e sinais**.

Diferentemente de uma MLP tradicional, a RNN possui **conexões recorrentes,** permitindo que informações de **etapas anteriores influenciem o processamento da etapa atual.**

Essa informação é mantida em um **estado oculto (*hidden state*),** que funciona como uma "memória" do que já foi processado na sequência.

Principal contribuição: as RNNs introduziram uma forma de **memória no processamento neural,** permitindo que a própria rede aprenda **relações, dependências e contexto ao longo da sequência.**

ARQUITETURAS DE REDES NEURAIS
RNN - RECURRENT NEURAL NETWORKS

Para que uma RNN consiga explorar a **ordem dos elementos,** a representação de entrada precisa **preservar a sequência**.

MLP com BoW / TF-IDF - Frase inteira → **um único vetor**

$$\text{“o gato subiu no telhado”} \rightarrow [x_1, x_2, \ldots, x_n]$$

A representação **não preserva explicitamente a ordem das palavras**.

RNN

Cada elemento da sequência precisa ser representado separadamente:

$$\text{“o”} \rightarrow v_1, \quad \text{“gato”} \rightarrow v_2, \quad \text{“subiu”} \rightarrow v_3, \ldots$$

- **Ideia central**

A RNN não recebe apenas “um vetor representando a frase”; ela recebe uma **sequência de vetores**, preservando a ordem de processamento.

ARQUITETURAS DE REDES NEURAIS
RNN – TIMESTEPS E ATRIBUTOS

Em uma RNN, **cada instância** é representada como uma **sequência de vetores.**

Então, imagine que cada documento seja representado por uma sequência de **50 vetores** e que cada vetor possua **100 atributos.**

Neste caso, a RNN receberá (N, 50, 100), onde:

- **N:** número de instâncias (“linhas” do dataset)
- **50:** quantidade de “passos” da sequência (formalmente chamado de **Timesteps**)
- **100:** atributos de cada vetor

Assim, em cada passo, a RNN recebe **um vetor de 100 valores.** A camada de entrada continua sem parâmetros treináveis, como na MLP.

RNN

1 Do conjunto de dados para a sequência de vetores

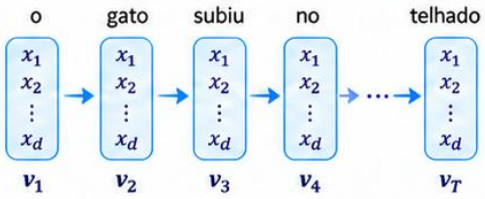
Conjunto de dados (exemplo)

id	texto
1	o gato subiu no telhado
2	ela leu
3	os alunos chegaram cedo para estudar na biblioteca
...	...

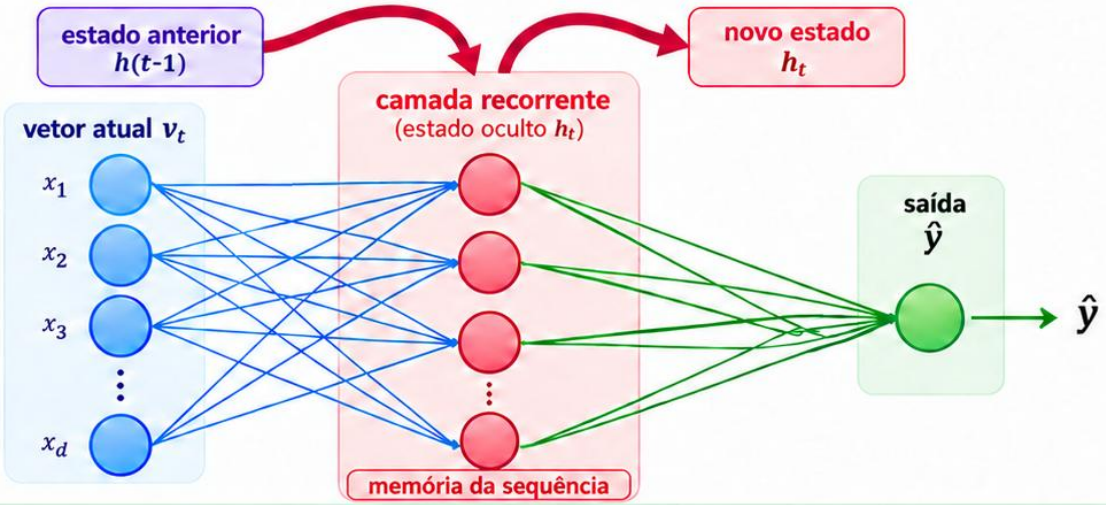
Instância selecionada

o gato subiu no telhado

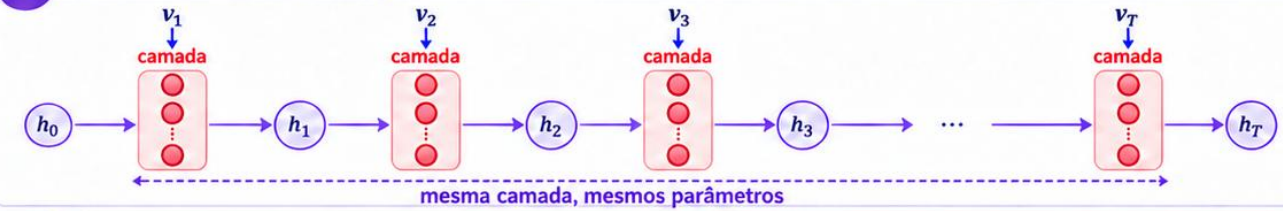
1 instância = sequência de vetores



2 Arquitetura da RNN



3 RNN desenrolada no tempo



1 Entrada sequencial
Cada instância é convertida em uma sequência de vetores $(v_1, v_2, ..., v_T)$.

2 Estado oculto carrega contexto
O estado oculto h_t funciona como uma memória da sequência.

3 Recorrência influencia o próximo passo
O estado h_t é passado para o próximo passo e influencia o processamento do próximo vetor da sequência.

Escolha de timesteps (T)

T define quantos tokens de cada documento serão processados pela rede

1 Descrição do produto

- Camiseta masculina de algodão
- Cor azul marinho
- Tamanho G
- Modelagem regular
- Instruções de lavagem
- Informações adicionais

Nem todo o texto precisa ter a mesma relevância.

2 Tokenização (unigramas) + vetores

Escolher **T = 50** significa processar até **50 tokens.**

Tokenização (unigramas): cada token = 1 palavra

camiseta	masculina	de	algodão	cor	azul	...
token 1	token 2	token 3	token 4	token 5	token 6	

Cada token → vetor de **100 atributos** (embedding)

v_1	v_2	v_3	v_4	v_5	v_6	...
(100 atributos)	(100 atributos)	(100 atributos)	(100 atributos)	(100 atributos)	(100 atributos)	

t1	t2	t3	...	t50	t51	t52	...

Processados pela RNN (T = 50) | Ignorados por truncamento

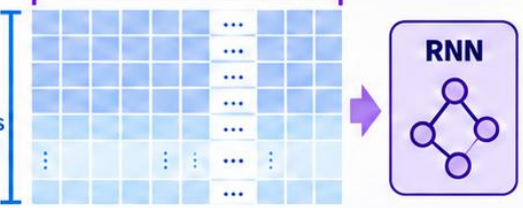
3 O que a RNN recebe

Matriz de entrada de um documento

50 timesteps × 100 atributos (embeddings)

100 atributos por token (embedding)

50 timesteps (tokens)



Documento curto → **padding**

Documento longo → **truncamento**

T deve ser escolhido com base nos dados.

T controla quantos tokens entram na sequência.

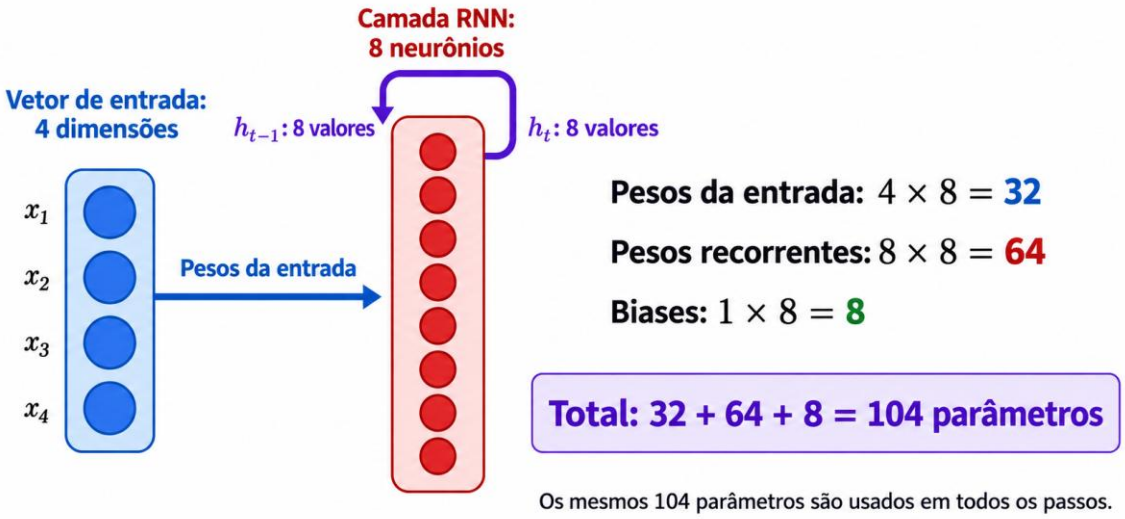
Cada token é uma **palavra** (unigrama) representada por um vetor de **100 atributos.**

A escolha deve considerar o **tipo** e o **tamanho** dos textos.

ARQUITETURAS DE REDES NEURAIS
RNN – PARÂMETROS

Uma camada RNN aprende pesos de entrada, pesos recorrentes e biases.

O exemplo abaixo mostra como calcular a quantidade desses parâmetros para uma entrada de 4 dimensões e uma camada com 8 neurônios.



ARQUITETURAS DE REDES NEURAIS
RNN – PARÂMETROS NAS RNN

Exercício: Uma **RNN simples** classifica documentos em **3 categorias**. Cada documento contém **10 vetores de 4 dimensões**.

A rede possui:

- **1ª camada recorrente:** 8 neurônios;
- **2ª camada recorrente:** 12 neurônios;
- **Camada de saída densa:** 3 neurônios, com *softmax*.

Considere **um bias por neurônio** e parâmetros reutilizados em todos os passos.

- **a)** Quantos pesos de entrada, pesos recorrentes e biases existem na **1ª camada**?
- **b)** Quantos pesos de entrada, pesos recorrentes e biases existem na **2ª camada**?
- **c)** Quantos pesos e biases existem na **camada de saída densa**?
- **d)** Qual é o **total de parâmetros treináveis** da rede?
- **e)** Se cada documento tivesse **20 passos**, o total de parâmetros mudaria? Justifique.

ARQUITETURAS DE REDES NEURAIS
RNN – PARÂMETROS NAS RNN

Resposta

a) 1ª camada recorrente
Entrada: 4 × 8 = 32 pesos
Recorrência: 8 × 8 = 64 pesos
Biases: 8
Total: 104 parâmetros
b) 2ª camada recorrente
Entrada: 8 × 12 = 96 pesos
Recorrência: 12 × 12 = 144 pesos
Biases: 12
Total: 252 parâmetros
c) Camada de saída densa
Pesos: 12 × 3 = 36
Biases: 3
Total: 39 parâmetros
d) Total: 104 + 252 + 39 = 395 parâmetros.
e) Não mudaria, pois os pesos e biases são reutilizados em todos os passos.

ARQUITETURAS DE REDES NEURAIS
RNN – ESTADO OCULTO (Ht)

Em uma **RNN simples,** a cada passo t, calcula-se primeiro o **vetor z_t,** combinando a entrada atual com o estado oculto anterior:

$$z_t = W_v v_t + W_h h_{\{t-1\}} + b$$

Em seguida, aplica-se a **função de ativação** para gerar o novo estado oculto:

$$h_t = f(z_t)$$

Onde:

- **v_t:** vetor atual da sequência.
- **h_{t-1}:** estado oculto anterior.
- **z_t:** vetor de valores calculados **antes da ativação,** como já vimos na MLP.
- **h_t:** novo estado oculto.
- **f:** função de ativação, geralmente **tanh**.

ARQUITETURAS DE REDES NEURAIS
RNN – LIMITAÇÕES DA RNN SIMPLES

A RNN utiliza o estado oculto *ht* para transportar informações ao longo da sequência de vetores que representam um documento. Em sequências longas, informações dos primeiros passos perdem influência sobre a saída.

Durante o *backpropagation through time*, o gradiente percorre muitos passos temporais. Multiplicações sucessivas tornam o gradiente muito pequeno (***vanishing gradient***), dificultando o aprendizado de dependências de longo prazo.

- **Consequência:** a RNN simples pode favorecer informações "recentes" e esquecer informações relevantes mais antigas.
- **Transição:** LSTM e GRU foram desenvolvidas para preservar informações por períodos mais longos ao longo do processamento de um documento.

ARQUITETURAS DE REDES NEURAIS
LSTM

LSTM (*Long Short-Term Memory*) é uma arquitetura **recorrente** desenvolvida para aprender dependências de **longo prazo.**

Ela acrescenta um **caminho específico** para transportar a **memória** ao longo da sequência e utiliza "**portas**" para controlar o fluxo de informações. As portas determinam o que deve ser **esquecido, armazenado** e **utilizado na saída.**

Essa estrutura reduz os efeitos do *vanishing gradient* durante o treinamento.

- **Ideia Central**

Preservar informações relevantes e descartar as que deixarem de ser úteis. Isso é feito a partir de uma informação de estado da célula que vai sendo mantido

- **Estado da Célula** (C_t)

Atua como uma esteira ao longo da cadeia, aplicando **pequenas interações lineares** para que a informação flua sem degradação (*vanishing gradient*) em sequências longas.

ARQUITETURAS DE REDES NEURAIS
LSTM - ESTADOS

A célula LSTM mantém **dois estados** entre os passos da sequência:

Estado da célula c_t (associado à memória de longo prazo)

- Transporta a memória interna ao longo da sequência.
- Preserva informações relevantes por mais tempo.
- É atualizado pelas portas de esquecimento e entrada.

Estado oculto h_t (associado à memória de curto prazo)

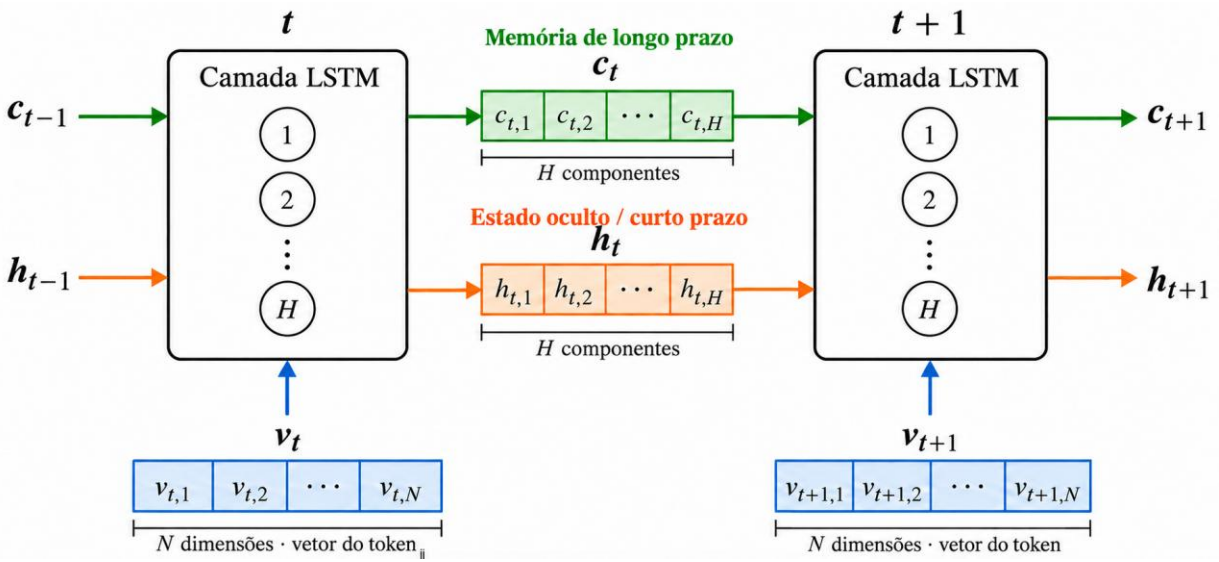
- Já existe na RNN simples.
- Representa a informação disponibilizada pela célula no passo atual.
- Participa do cálculo da saída e das portas do próximo passo.

ARQUITETURAS DE REDES NEURAIS
LSTM - ESTADOS

Ambos os valores c_t e h_t seguem para o próximo passo.

A cada instante t, a LSTM recebe x_t, h_{t-1} e c_{t-1}, produzindo h_t e c_t.

A quantidade de "passos" corresponde à quantidade de vetores utilizados para representar o documento, com **um vetor processado por passo**. Essa característica **recorrente**, típica da RNN simples, permanece na LSTM.



ARQUITETURAS DE REDES NEURAIS
LSTM – FORGET GATE

A **porta de esquecimento** controla quanto da memória $\mathbf{c_{t-1}}$ será preservado.

- **Cálculo com pesos e bias:** $z_f = W_f v_t + U_f h_{\{t-1\}} + b_f$
- **Aplicação da sigmoide:** $f_t = \sigma(z_f)$
- **Filtragem da memória anterior, elemento a elemento:** $\boldsymbol{f_t} \odot \boldsymbol{c_{\{t-1\}}}$

Cada componente de $\mathbf{f_t}$ varia entre **0 e 1**: próximo de **0, reduz a memória**; próximo de **1, preserva**; valores intermediários preservam parcialmente.

$\mathbf{c_{t-1}}$ não participa do cálculo de z_f: é filtrado depois.

A informação nova será acrescentada na atualização da memória.

ARQUITETURAS DE REDES NEURAIS
LSTM – INPUT GATE

A **porta de entrada** controla quanto da **informação candidata $\mathbf{g_t}$** será acrescentado à memória.

- **Cálculo com pesos e bias:** $z_i = W_i v_t + U_i h_{t-1} + b_i$
- **Aplicação da sigmoide:** $i_t = \sigma(z_i)$
- **Filtragem da informação candidata, elemento a elemento:** $\boldsymbol{i_t} \odot \boldsymbol{g_t}$

Cada componente de $\boldsymbol{i_t}$ varia entre **0 e 1**: próximo de **0,** bloqueia a informação candidata; próximo de **1,** permite sua incorporação; valores intermediários permitem parcialmente.

$\mathbf{g_t}$ é calculado separadamente, com pesos e bias próprios e ativação tanh.

A parcela $\boldsymbol{i_t} \odot \boldsymbol{g_t}$ será somada à memória anterior já filtrada.

ARQUITETURAS DE REDES NEURAIS
LSTM – CANDIDATE GATE

A **informação candidata $\mathbf{g_t}$** representa o conteúdo proposto para ser acrescentado à memória. **Ela não é uma porta.**

- **Cálculo com pesos e bias próprios:** $z_g = W_g v_t + U_g h_{t-1} + b_g$
- **Aplicação da tangente hiperbólica:** $g_t = \tanh(z_g)$

Cada componente de $\mathbf{g_t}$ varia entre **−1 e 1,** podendo contribuir positiva ou negativamente para a memória.

A porta de entrada $\mathbf{i_t}$ controla quanto dessa informação será incorporado, pela multiplicação elemento a elemento $\mathbf{i_t} \odot \mathbf{g_t}$.

ARQUITETURAS DE REDES NEURAIS
LSTM – OUTPUT GATE

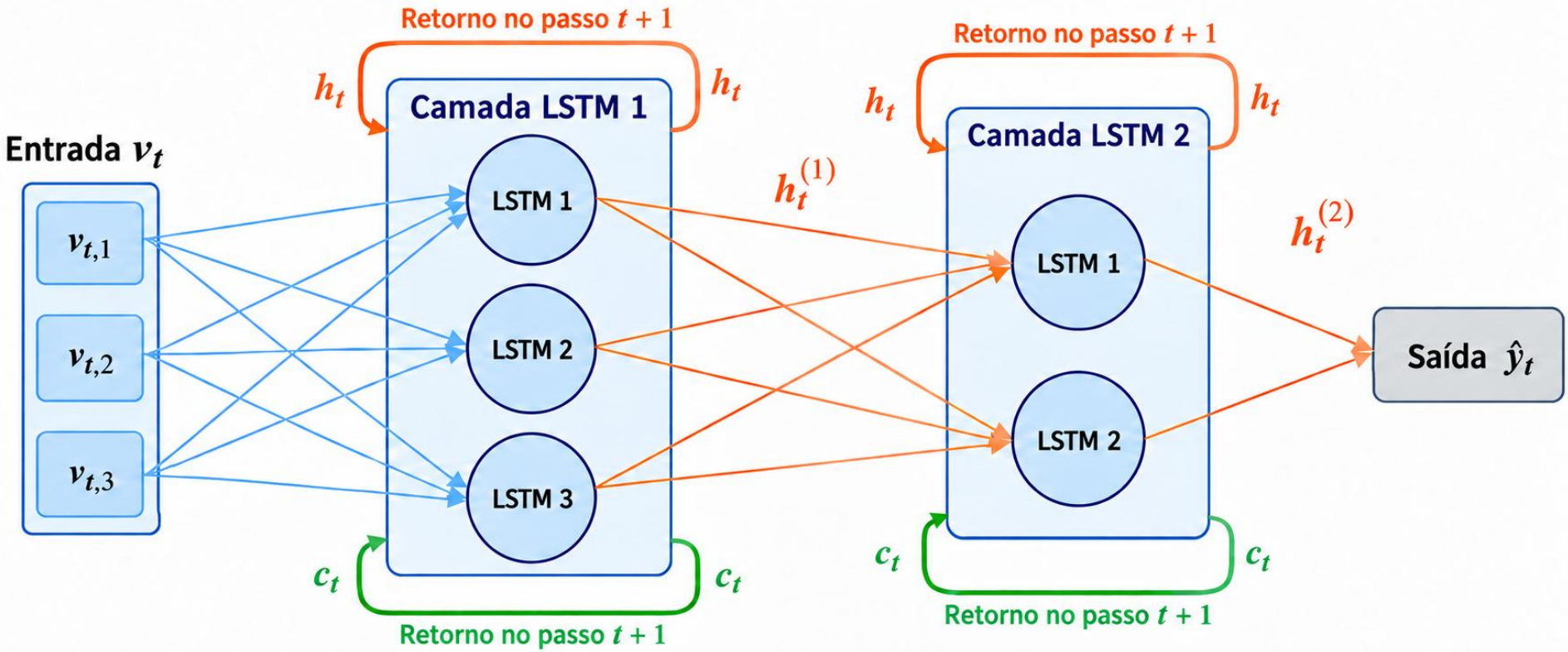
A **porta de saída (*Output gate*)** controla quanto da memória transformada por **tanh** será utilizado para gerar $\mathbf{h_t}$.

- **Cálculo com pesos e bias próprios:** $z_o = W_o v_t + U_o h_{t-1} + b_o$
- **Aplicação da sigmoide:** $o_t = \sigma(z_o)$
- **Geração do estado oculto, elemento a elemento:** $h_t = o_t \odot \tanh(c_t)$

Cada componente de $\mathbf{o_t}$ varia entre **0 e 1**: próximo de **0,** bloqueia sua contribuição para h_t; próximo de **1,** permite; valores intermediários permitem parcialmente.

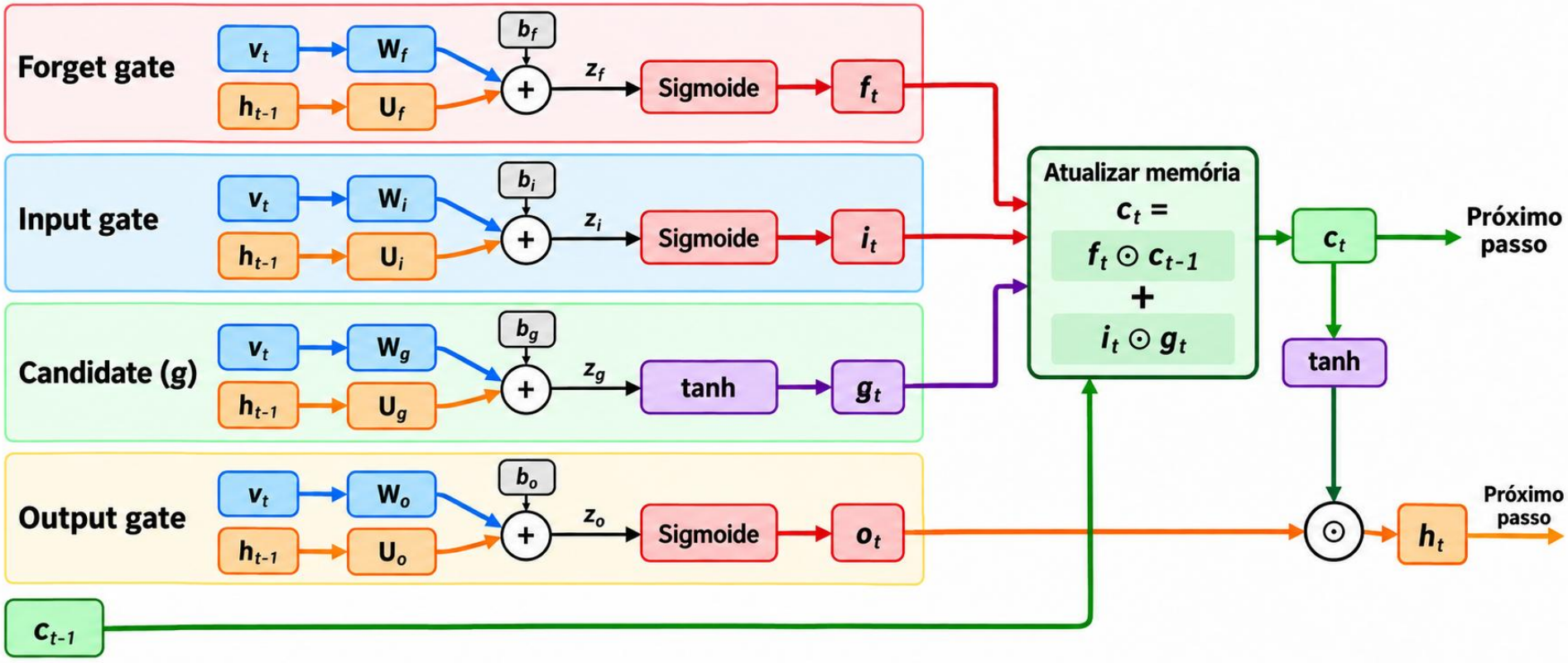
Essa operação não altera c_t. A memória c_t e o estado oculto h_t seguem para o próximo passo.

Rede com duas camadas ocultas LSTM



h_t e c_t gerados no passo t retornam como estados anteriores no passo $t+1$.

LSTM: quatro cálculos em cada passo



ARQUITETURAS DE REDES
FLUXO COMPLETO

Entradas: v_t, h_{t-1} e c_{t-1}

1. Filtrar a memória anterior

$$f_t = \sigma\left(W_f v_t + U_f h_{t-1} + b_f\right)$$

2. Calcular a entrada e a informação candidata

$$i_t = \sigma(W_i v_t + U_i h_{t-1} + b_i)$$
$$g_t = \tanh\left(W_g v_t + U_g h_{t-1} + b_g\right)$$

3. Atualizar a memória

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

4. Gerar o novo estado oculto

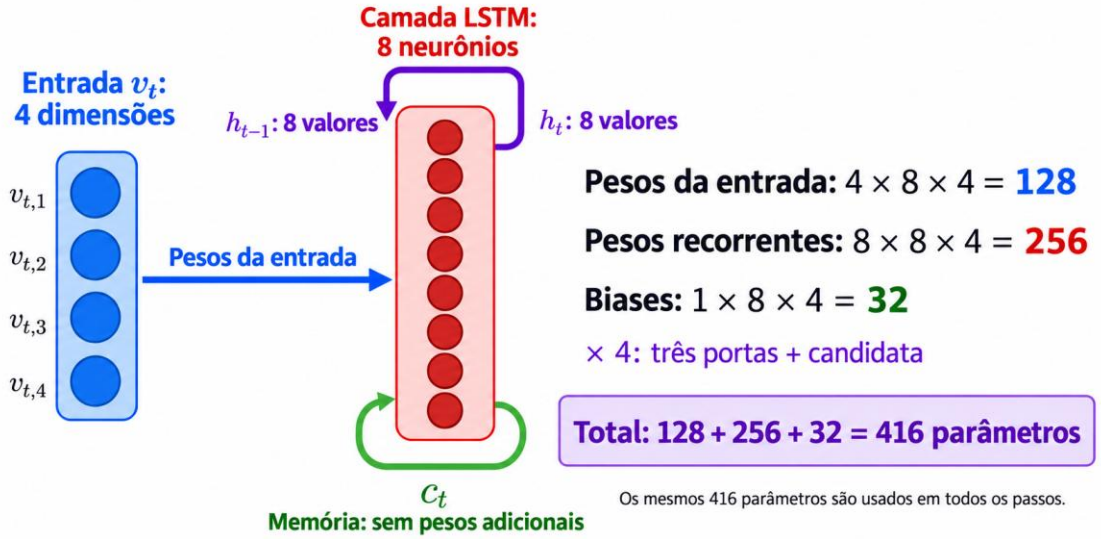
$$o_t = \sigma(W_o v_t + U_o h_{t-1} + b_o)$$
$$h_t = o_t \odot \tanh(c_t)$$

Saídas: c_t e h_t, que seguem para o próximo passo.

ARQUITETURAS DE REDES NEURAIS
LSTM – PARÂMETROS

A **LSTM é um tipo de RNN** que aprende **quatro conjuntos de pesos de entrada, pesos recorrentes e biases**: um para cada porta e outro para a informação candidata.

O exemplo abaixo mostra como calcular a quantidade desses parâmetros para uma **entrada de 4 dimensões** e uma **camada com 8 neurônios.**



ARQUITETURAS DE REDES NEURAIS
LSTM – PARÂMETROS NAS LSTM

Exemplo: Uma **LSTM** classifica documentos em **3 categorias.** Cada documento contém **10 vetores de 4 dimensões.**

A rede possui:

- **1ª camada LSTM:** 8 neurônios;
- **2ª camada LSTM:** 12 neurônios;
- **Camada de saída densa:** 3 neurônios, com *softmax*, recebendo o último estado oculto.

Considere **quatro conjuntos de parâmetros por neurônio LSTM** (três portas e a informação candidata), com **um bias por conjunto,** reutilizados em todos os passos.

- **a)** Quantos pesos de entrada, pesos recorrentes e biases existem na **1ª camada LSTM**?
- **b)** Quantos pesos de entrada, pesos recorrentes e biases existem na **2ª camada LSTM**?
- **c)** Quantos pesos e biases existem na **camada de saída densa**?
- **d)** Qual é o **total de parâmetros treináveis** da rede?
- **e)** Se cada documento tivesse **20 passos,** o total de parâmetros mudaria? Justifique.

ARQUITETURAS DE REDES NEURAIS
LSTM – PARÂMETROS NAS LSTM

Resposta

a) 1ª camada LSTM
Entrada: $4 \times 8 \times 4 = 128$ pesos
Recorrência: $8 \times 8 \times 4 = 256$ pesos
Biases: $8 \times 4 = 32$
Total: 416 parâmetros
b) 2ª camada LSTM
Entrada: $8 \times 12 \times 4 = 384$ pesos
Recorrência: $12 \times 12 \times 4 = 576$ pesos
Biases: $12 \times 4 = 48$
Total: 1.008 parâmetros
c) Camada de saída densa
Pesos: $12 \times 3 = 36$
Biases: 3
Total: 39 parâmetros
d) Total: $416 + 1.008 + 39 = 1.463$ parâmetros.
e) Não mudaria, pois os mesmos pesos e biases são reutilizados em todos os passos.